

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Fundamentals of Data Science

COURSE CODE: DSA04

COURSE OUTCOME COVERED

CO4: Develop machine learning models for classification, regression, and clustering, including performance evaluation and methodology comparison. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

**Annexure A
Industry Problem Solving**

Code of Conduct:

I Shaik Mohammed Abrar / 192472185 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Shaik Mohammed Abrar

RUBRICS FOR CALCULATION

SCENARIO BASED ASSESSMENT							
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Level	Marks
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario		
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues		
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts		
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided		
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

Annexure A

Question 1: Real Estate Price Prediction Using Linear Regression

A real estate company wants to predict the **selling price of houses** based on house-related features.

House	Area sq.ft	Bedrooms	Age	Distance from City km	Price Lakhs
H1	900	2	12	10	45
H2	1100	2	10	8	55
H3	1300	3	8	6	68
H4	1500	3	5	5	82
H5	1800	4	3	3	115
H6	2000	4	2	2	135
H7	1000	2	15	12	40
H8	1600	3	6	4	90

Tasks

- Create the dataset using Python.
- Identify independent and dependent variables.
- Split the dataset into training and testing data.
- Implement **Linear Regression**.
- Predict house price for test data.
- Predict the price of a new house with:

Area	Bedrooms	Age	Distance
1700	3	4	4

- Evaluate the model using **MAE, MSE, RMSE, and R² score**.
- Interpret the result and explain whether the model is useful for real estate price prediction.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
data = {  
    "House": ["H1", "H2", "H3", "H4", "H5", "H6", "H7", "H8"],  
    "Area": [900, 1100, 1300, 1500, 1800, 2000, 1000, 1600],  
    "Bedrooms": [2, 2, 3, 3, 4, 4, 2, 3],  
    "Age": [12, 10, 8, 5, 3, 2, 15, 6],  
    "Distance": [10, 8, 6, 5, 3, 2, 12, 4],  
    "Price": [45, 55, 68, 82, 115, 135, 40, 90]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
X = df[["Area", "Bedrooms", "Age", "Distance"]]
```

```
y = df["Price"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42  
)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("\nActual Prices:")
```

```
print(y_test.values)
```

```
print("\nPredicted Prices:")
```

```
print(np.round(y_pred, 2))
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("\nModel Evaluation:")
```

```
print("MAE:", round(mae, 4))
```

```
print("MSE:", round(mse, 4))
```

```
print("RMSE:", round(rmse, 4))
```

```
print("R2 Score:", round(r2, 4))
```

```
new_house = pd.DataFrame({
```

```
    "Area": [1700],
```

```
    "Bedrooms": [3],
```

```
    "Age": [4],
```

```
    "Distance": [4]
```

```
})
```

```
new_price = model.predict(new_house)
```

```
print("\nNew House Details:")
```

```
print(new_house)
```

```
print("\nPredicted Price for New House:")
```

```
print(round(new_price[0], 2), "Lakhs")
```

```
print("\nModel Coefficients:")
```

```
for feature, coefficient in zip(X.columns, model.coef_):
```

```
    print(feature, ":", round(coefficient, 4))
```

```
print("\nIntercept:", round(model.intercept_, 4))
```

```
plt.figure(figsize=(7, 5))
```

```
plt.scatter(y_test, y_pred, s=80)
```

```
plt.xlabel("Actual Price (Lakhs)")
```

```
plt.ylabel("Predicted Price (Lakhs)")
```

```
plt.title("Actual vs Predicted House Prices")
```

```
plt.grid(True)
```

```
plt.show()
```

```
plt.figure(figsize=(7, 5))
```

```
plt.bar(X.columns, model.coef_)
```

```
plt.xlabel("Features")
```

```
plt.ylabel("Coefficient")
```

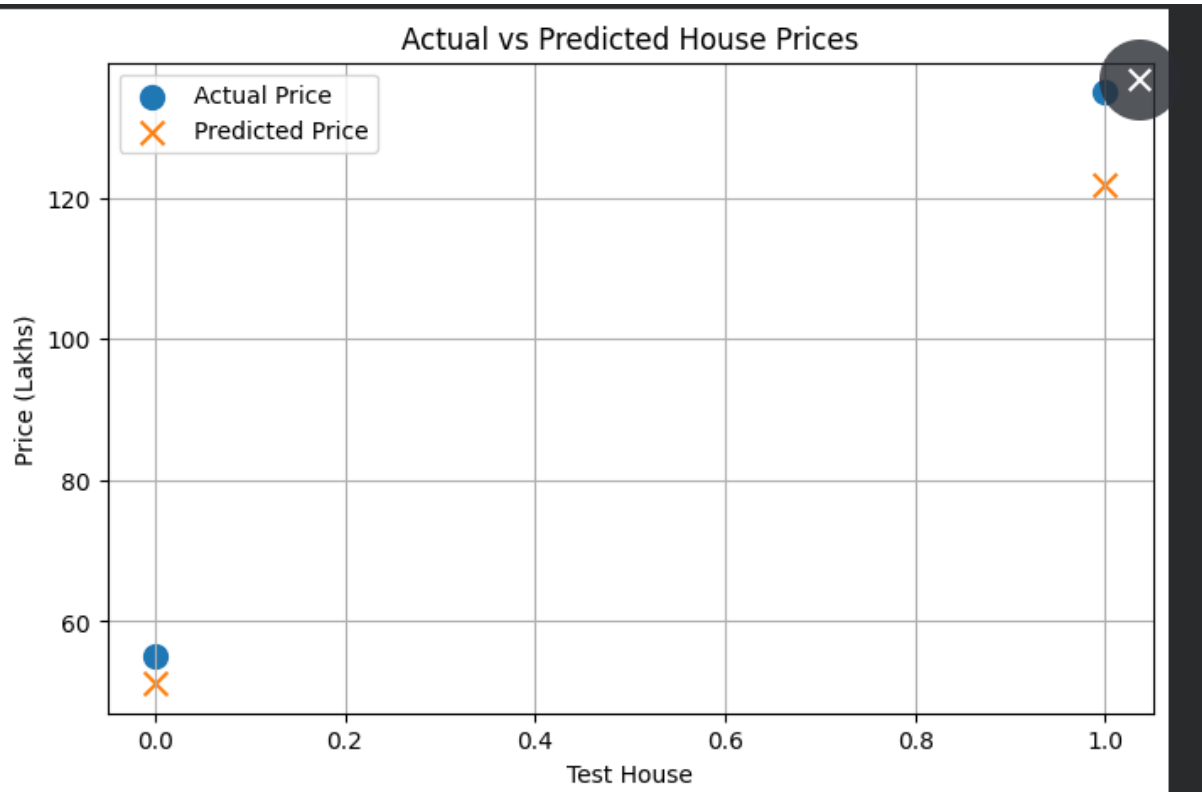
```
plt.title("Linear Regression Coefficients")
```

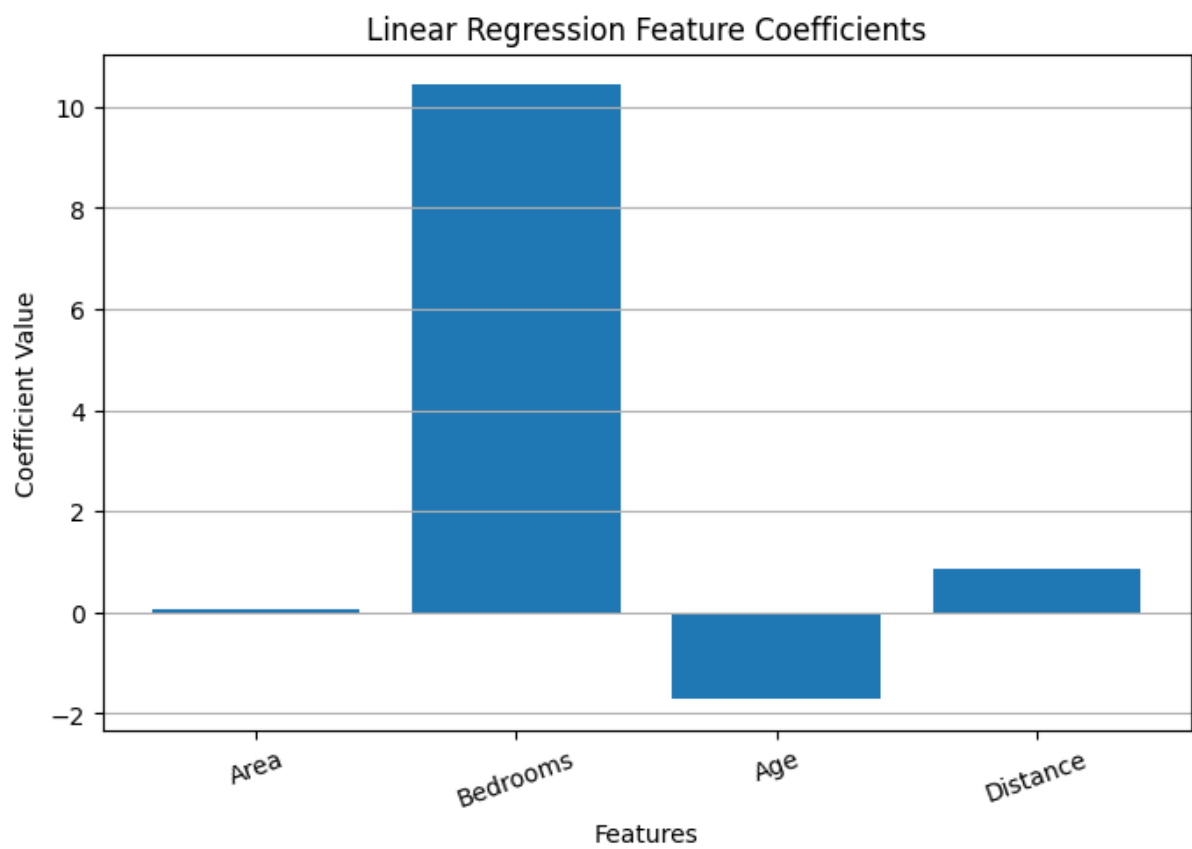
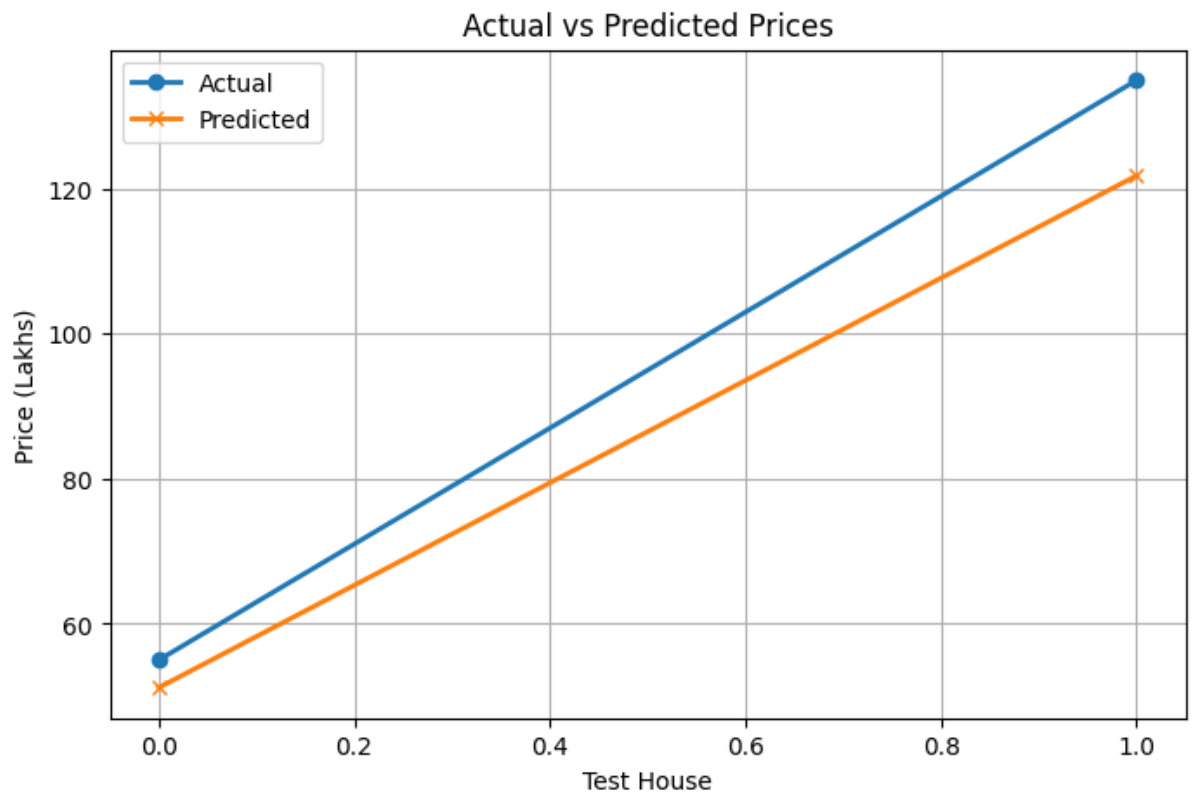
```
plt.xticks(rotation=20)
```

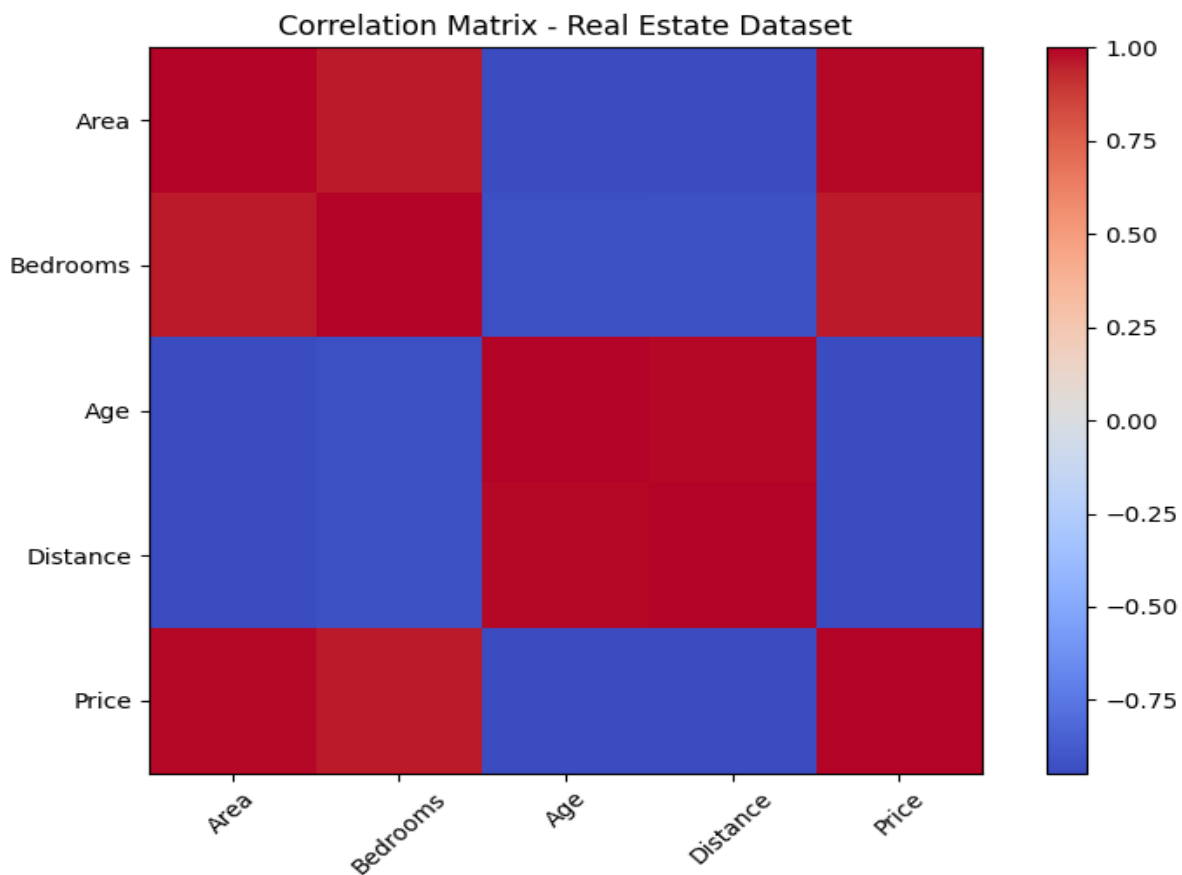
```
plt.grid(axis="y")
```

```
plt.show()
```

Output:







Question 2: Customer Churn Prediction Using Logistic Regression

A telecom company wants to predict whether a customer will **churn** or **not churn**.

Customer	Monthly Bill	Complaints	Data Usage GB	Tenure Months	Churn
C1	500	0	25	36	0
C2	900	3	10	8	1
C3	750	2	15	12	1
C4	400	0	30	48	0
C5	1000	4	8	6	1
C6	550	1	22	30	0
C7	850	3	12	10	1
C8	450	0	28	40	0

Here,

Churn = 1 means customer leaves the company.

Churn = 0 means customer stays.

Tasks

- a) Create the dataset using Python.
- b) Identify the input features and target variable.
- c) Perform train-test split.
- d) Implement **Logistic Regression**.
- e) Predict churn status for test customers.
- f) Predict churn for a new customer:

Monthly Bill	Complaints	Data Usage GB	Tenure Months
800	2	14	10

- g) Evaluate the model using **accuracy, confusion matrix, precision, recall, and F1-score**.
- h) Explain why Logistic Regression is suitable for churn prediction.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score,
f1_score, ConfusionMatrixDisplay
```

```
data = {
```

```
    "Customer": ["C1", "C2", "C3", "C4", "C5", "C6", "C7", "C8"],
```

```
    "Monthly_Bill": [500, 900, 750, 400, 1000, 550, 850, 450],
```

```
    "Complaints": [0, 3, 2, 0, 4, 1, 3, 0],
```

```
    "Data_Usage_GB": [25, 10, 15, 30, 8, 22, 12, 28],
```

```
    "Tenure_Months": [36, 8, 12, 48, 6, 30, 10, 40],
```

```
    "Churn": [0, 1, 1, 0, 1, 0, 1, 0]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
X = df[["Monthly_Bill", "Complaints", "Data_Usage_GB", "Tenure_Months"]]
```

```
y = df["Churn"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.25, random_state=42, stratify=y
```

```
)
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
y_prob = model.predict_proba(X_test)[:, 1]
```

```
print("\nActual Churn:")
```

```
print(y_test.values)
```

```
print("\nPredicted Churn:")
```

```
print(y_pred)
```

```
print("\nChurn Probability:")
```

```
print(np.round(y_prob, 2))
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred, zero_division=0)
```

```
recall = recall_score(y_test, y_pred, zero_division=0)
```

```
f1 = f1_score(y_test, y_pred, zero_division=0)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nModel Evaluation:")
```

```
print("Accuracy:", round(accuracy, 4))
```

```
print("Precision:", round(precision, 4))
```

```
print("Recall:", round(recall, 4))
```

```
print("F1 Score:", round(f1, 4))
```

```
print("\nConfusion Matrix:")
```

```
print(cm)
```

```
print("\nModel Coefficients:")
```

```
for feature, coefficient in zip(X.columns, model.coef_[0]):
```

```
    print(feature, ":", round(coefficient, 4))
```

```
print("\nIntercept:", round(model.intercept_[0], 4))
```

```
new_customer = pd.DataFrame({
```

```
    "Monthly_Bill": [800],
```

```
    "Complaints": [2],
```

```
    "Data_Usage_GB": [14],
```

```
    "Tenure_Months": [10]
```

```
})
```

```
new_prediction = model.predict(new_customer)
```

```
new_probability = model.predict_proba(new_customer)[0][1]
```

```
print("\nNew Customer:")
```

```
print(new_customer)
```

```
print("\nChurn Probability:", round(new_probability * 100, 2), "%")
```

```
if new_prediction[0] == 1:
```

```
    print("Predicted Status: Churn")
```

```
else:
```

```
    print("Predicted Status: Stay")
```

```
disp = ConfusionMatrixDisplay(
```

```
    confusion_matrix=cm,
```

```
    display_labels=["Stay", "Churn"]
```

```
)
```

```
disp.plot(values_format="d")
```

```
plt.title("Confusion Matrix - Customer Churn")
```

```
plt.show()
```

```
plt.figure(figsize=(7, 5))
```

```
plt.bar(X.columns, model.coef_[0])
```

```
plt.xlabel("Features")

plt.ylabel("Coefficient")

plt.title("Logistic Regression Coefficients")

plt.xticks(rotation=20)

plt.grid(axis="y")

plt.show()
```

```
plt.figure(figsize=(7, 5))

plt.scatter(df["Monthly_Bill"], df["Churn"], s=80)

plt.xlabel("Monthly Bill")

plt.ylabel("Churn")

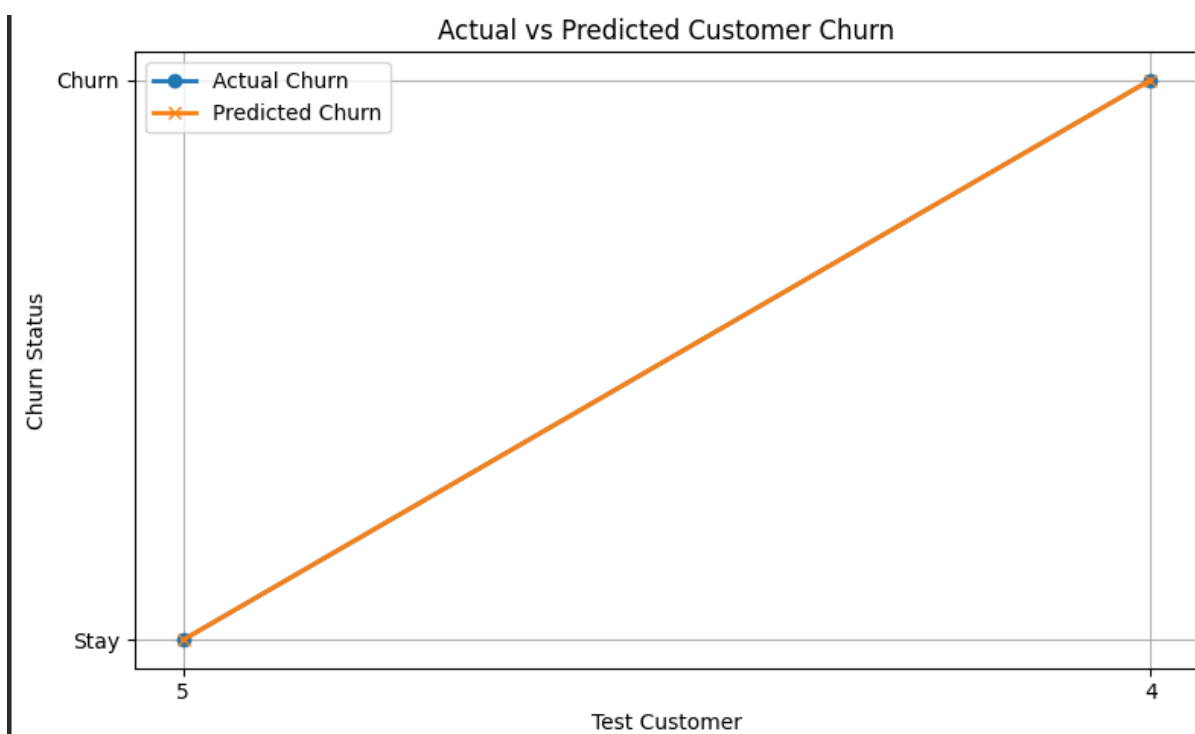
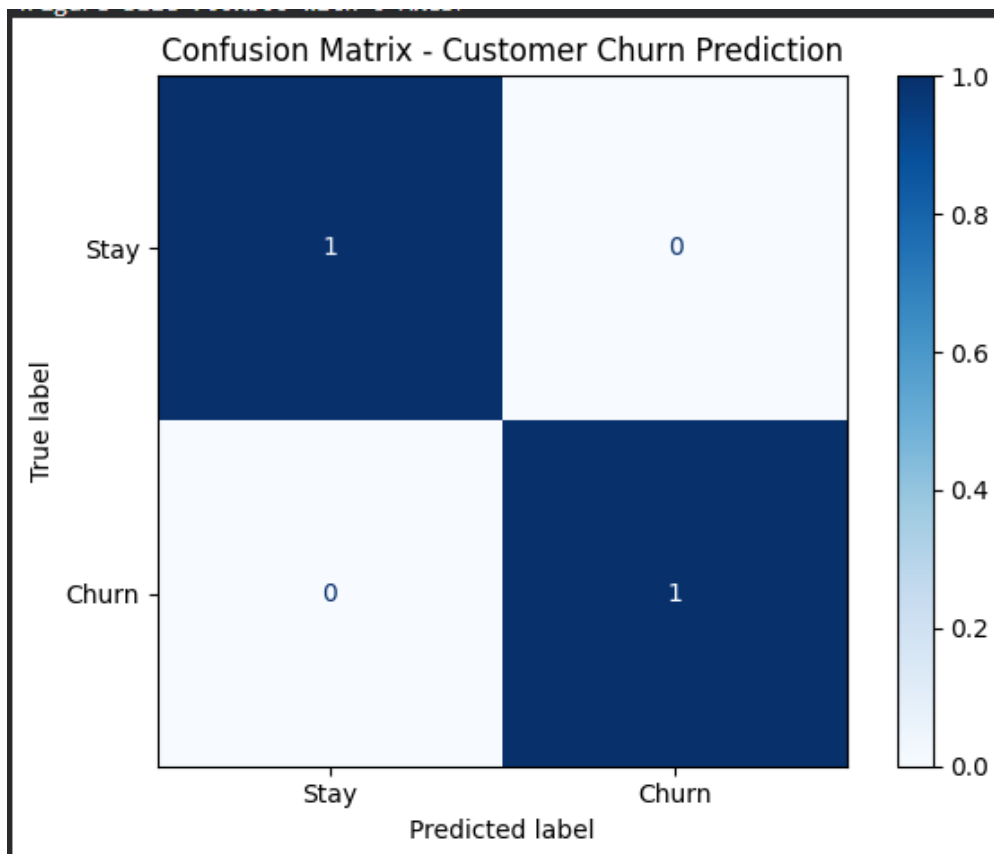
plt.title("Monthly Bill vs Customer Churn")

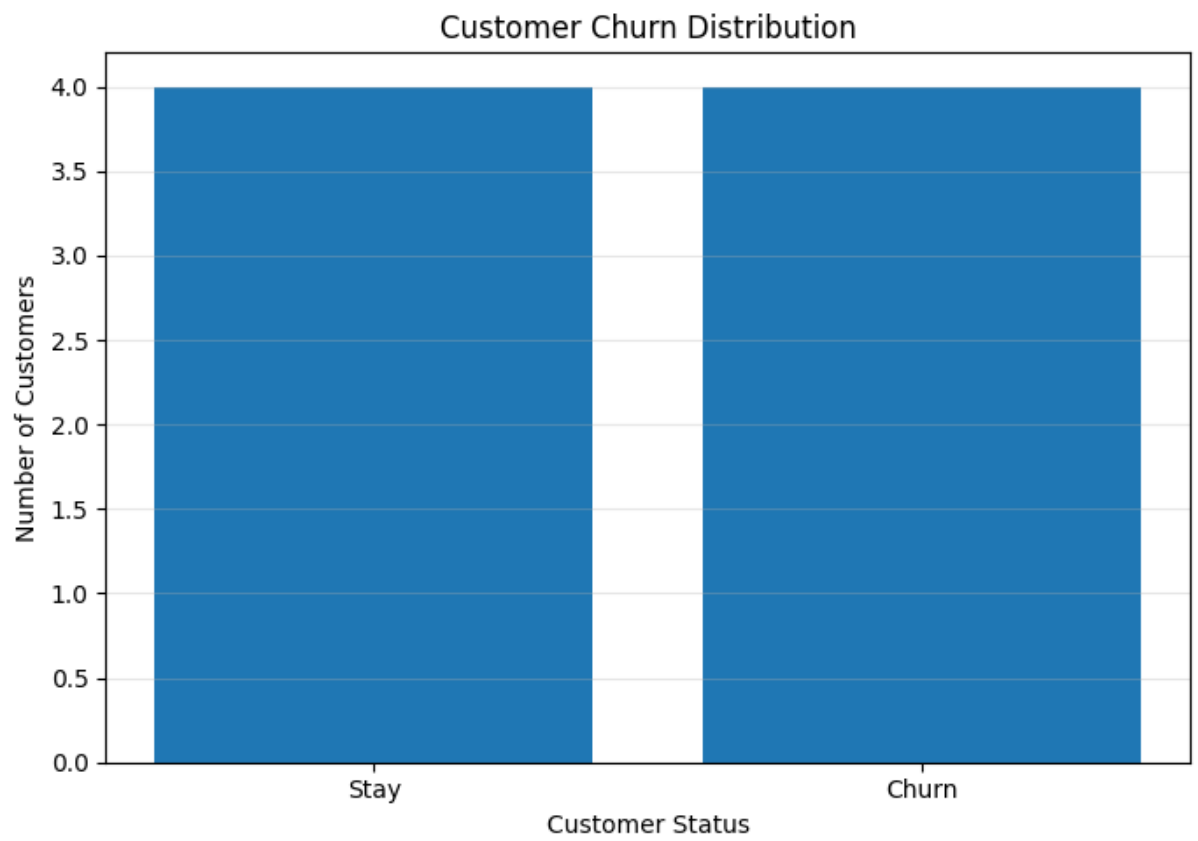
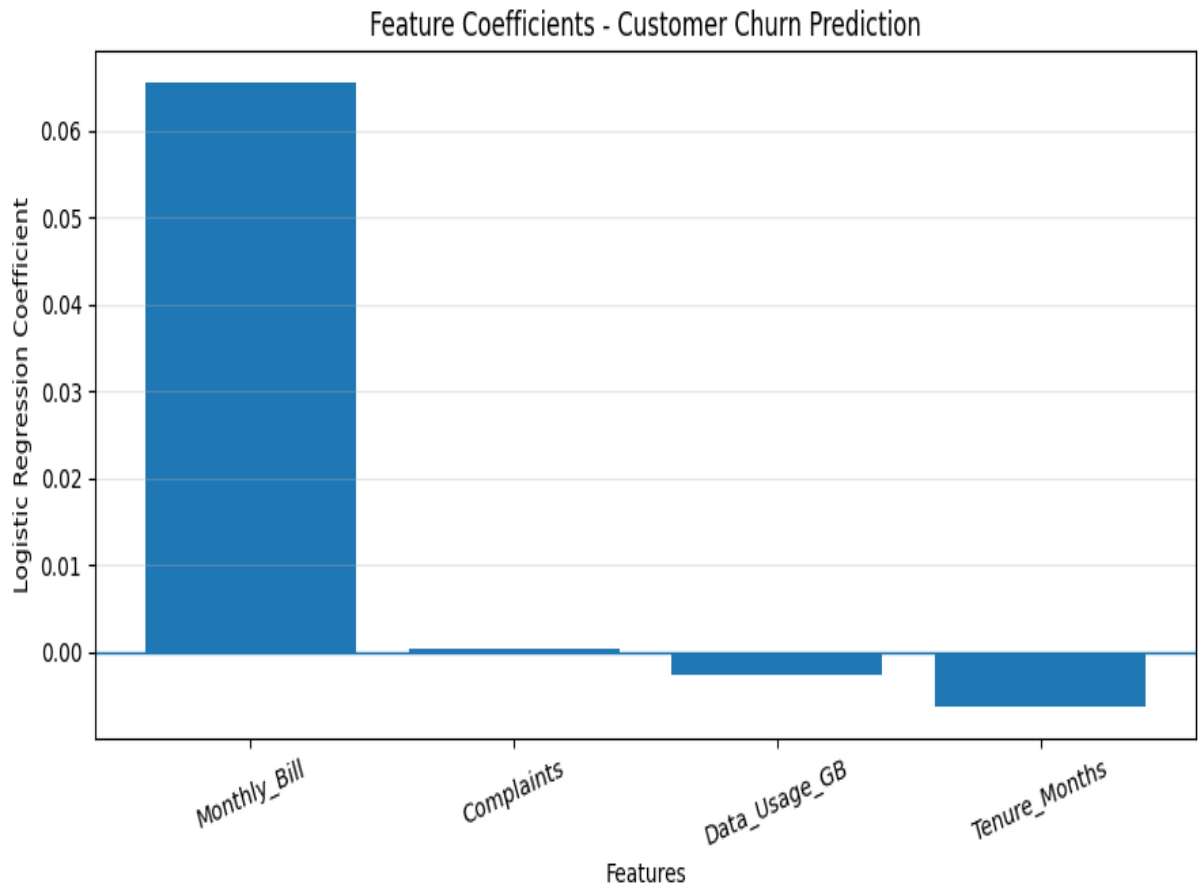
plt.yticks([0, 1], ["Stay", "Churn"])

plt.grid(True)

plt.show()
```

Output:





Question 3: Marketing Sales Prediction Using Lasso Regression

A retail company wants to predict **monthly sales** based on advertisement spending across different platforms. The company wants to know which marketing channels are most important.

Month	TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns	Sales Lakhs
M1	50	20	10	5	15
M2	60	25	12	6	18
M3	80	40	15	8	25
M4	100	60	20	10	32
M5	120	70	22	12	38
M6	40	15	8	4	12
M7	90	50	18	9	28
M8	110	65	21	11	35

Tasks

- Create the dataset using Python.
- Identify features and target variable.
- Apply train-test split.
- Implement **Lasso Regression**.
- Use an alpha value such as **0.1 or 1.0**.
- Predict sales for test data.
- Predict sales for a new campaign:

TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns
95	55	18	9

- Evaluate using **MAE, MSE, RMSE, and R² score**.
- Display the coefficients and identify which features are reduced or selected by Lasso.
- Explain how Lasso helps in feature selection.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import Lasso
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
data = {  
    "Month": ["M1", "M2", "M3", "M4", "M5", "M6", "M7", "M8"],  
    "TV_Ads": [50, 60, 80, 100, 120, 40, 90, 110],  
    "Social_Media_Ads": [20, 25, 40, 60, 70, 15, 50, 65],  
    "Newspaper_Ads": [10, 12, 15, 20, 22, 8, 18, 21],  
    "Email_Campaigns": [5, 6, 8, 10, 12, 4, 9, 11],  
    "Sales": [15, 18, 25, 32, 38, 12, 28, 35]  
}  
  
df = pd.DataFrame(data)  
  
print(df)  
  
X = df[["TV_Ads", "Social_Media_Ads", "Newspaper_Ads", "Email_Campaigns"]]  
y = df["Sales"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42  
)  
  
model = Lasso(alpha=0.1, max_iter=10000)  
model.fit(X_train, y_train)  
  
y_pred = model.predict(X_test)
```

```
print("\nActual Sales:")

print(y_test.values)


print("\nPredicted Sales:")

print(np.round(y_pred, 2))


mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)


print("\nModel Evaluation:")

print("MAE:", round(mae, 4))

print("MSE:", round(mse, 4))

print("RMSE:", round(rmse, 4))

print("R2 Score:", round(r2, 4))


print("\nLasso Coefficients:")

for feature, coefficient in zip(X.columns, model.coef_):

    print(feature, ":", round(coefficient, 4))


print("\nIntercept:", round(model.intercept_, 4))


new_campaign = pd.DataFrame({

    "TV_Ads": [95],
```

```
"Social_Media_Ads": [55],  
"Newspaper_Ads": [18],  
"Email_Campaigns": [9]  
})
```

```
new_sales = model.predict(new_campaign)
```

```
print("\nNew Campaign:")
```

```
print(new_campaign)
```

```
print("\nPredicted Sales:", round(new_sales[0], 2), "Lakhs")
```

```
plt.figure(figsize=(7, 5))
```

```
plt.scatter(y_test, y_pred, s=80)
```

```
plt.xlabel("Actual Sales")
```

```
plt.ylabel("Predicted Sales")
```

```
plt.title("Actual vs Predicted Sales")
```

```
plt.grid(True)
```

```
plt.show()
```

```
plt.figure(figsize=(8, 5))
```

```
plt.bar(X.columns, model.coef_)
```

```
plt.xlabel("Marketing Features")
```

```
plt.ylabel("Lasso Coefficient")
```

```
plt.title("Lasso Regression Coefficients")
```

```
plt.xticks(rotation=20)
```

```
plt.grid(axis="y")
```

```
plt.show()
```

```
plt.figure(figsize=(7, 5))
```

```
plt.scatter(df["TV_Ads"], df["Sales"], s=80)
```

```
plt.xlabel("TV Ads")
```

```
plt.ylabel("Sales")
```

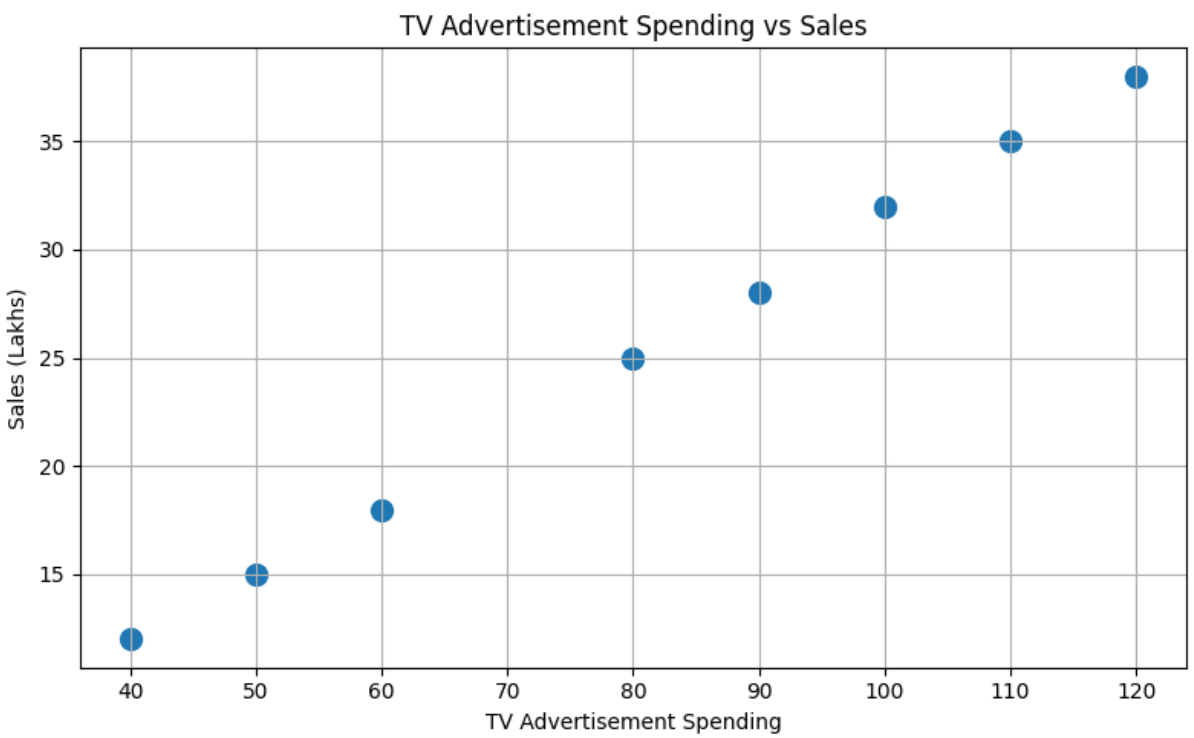
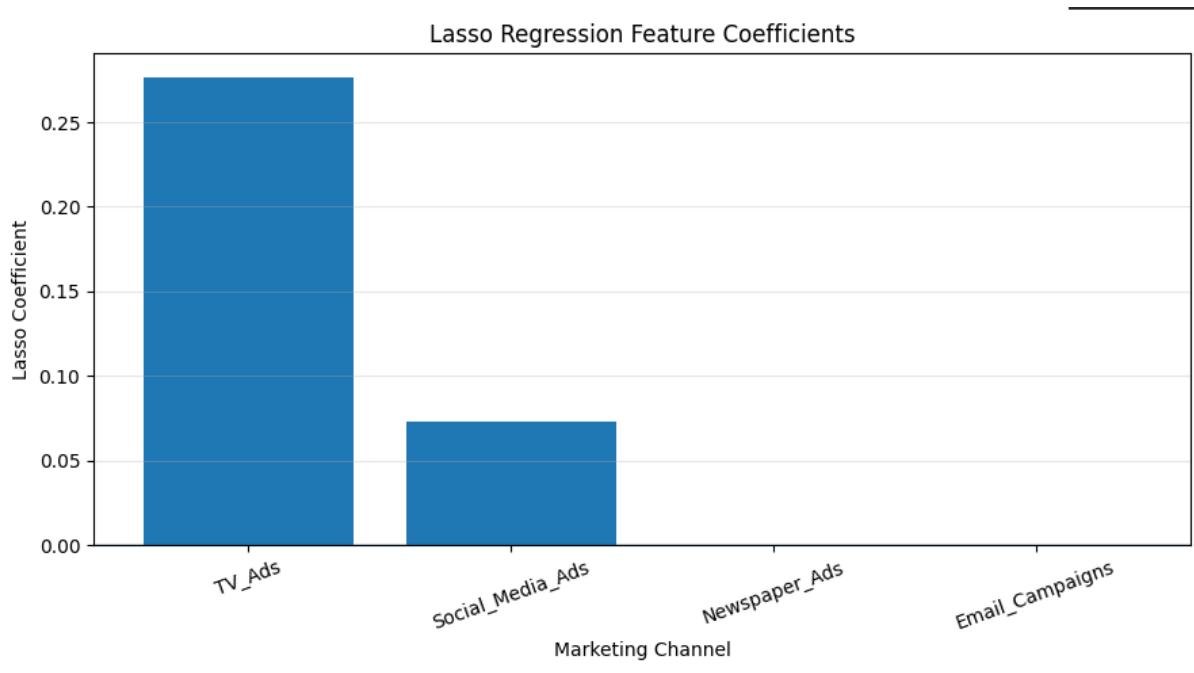
```
plt.title("TV Ads vs Sales")
```

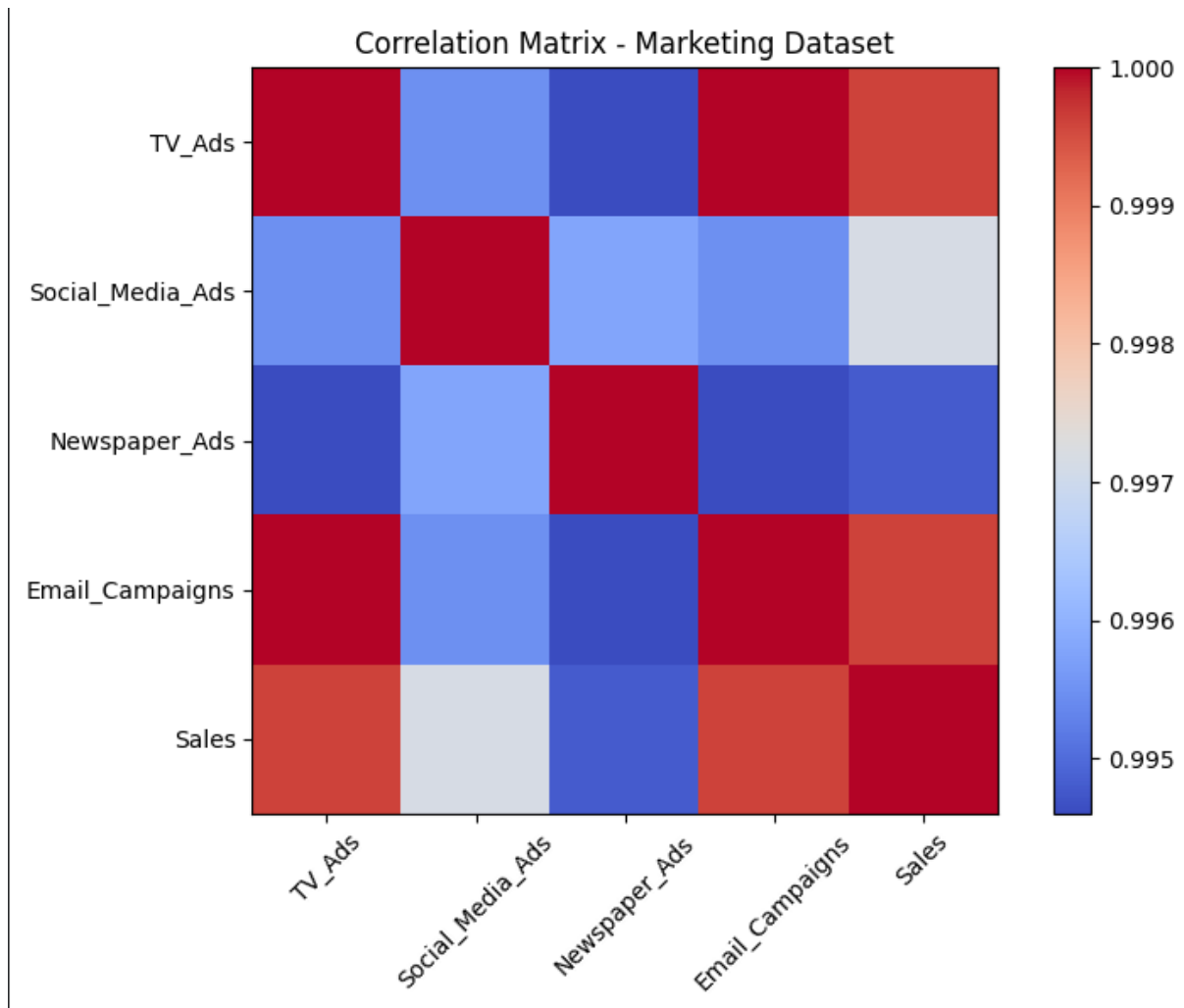
```
plt.grid(True)
```

```
plt.show()
```

Output:







Question 4: Electricity Consumption Prediction Using Ridge Regression

A smart energy company wants to predict household **electricity consumption**. Some features may be highly correlated, such as number of appliances and daily usage hours. Therefore, Ridge Regression is used.

House	Appliances	Usage Hours	House Size sq.ft	Occupants	Consumption Units
H1	5	4	800	3	120
H2	7	6	1000	4	190
H3	8	7	1200	4	240
H4	10	8	1500	5	320
H5	4	3	700	2	90
H6	6	5	900	3	160
H7	9	7	1300	5	280
H8	11	9	1600	6	350

Tasks

- a) Create the dataset using Python.
- b) Identify the independent and dependent variables.
- c) Split the data into training and testing sets.
- d) Implement **Ridge Regression**.
- e) Use an alpha value such as **1.0**.
- f) Predict electricity consumption for test data.
- g) Predict consumption for a new house:

Appliances	Usage Hours	House Size	Occupants
8	6	1100	4

- h) Evaluate using **MAE, MSE, RMSE, and R² score**.
- i) Display the Ridge coefficients.
- j) Explain how Ridge Regression reduces overfitting.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Ridge

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


data = {

    "House": ["H1", "H2", "H3", "H4", "H5", "H6", "H7", "H8"],

    "Appliances": [5, 7, 8, 10, 4, 6, 9, 11],

    "Usage_Hours": [4, 6, 7, 8, 3, 5, 7, 9],

    "House_Size": [800, 1000, 1200, 1500, 700, 900, 1300, 1600],

    "Occupants": [3, 4, 4, 5, 2, 3, 5, 6],

    "Consumption": [120, 190, 240, 320, 90, 160, 280, 350]

}


df = pd.DataFrame(data)
```



```
print(df)
```

```
X = df[["Appliances", "Usage_Hours", "House_Size", "Occupants"]]
```

```
y = df["Consumption"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.25, random_state=42
```

```
)
```

```
model = Ridge(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("\nActual Consumption:")
```

```
print(y_test.values)
```

```
print("\nPredicted Consumption:")
```

```
print(np.round(y_pred, 2))
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("\nModel Evaluation:")

print("MAE:", round(mae, 4))

print("MSE:", round(mse, 4))

print("RMSE:", round(rmse, 4))

print("R2 Score:", round(r2, 4))


print("\nRidge Coefficients:")

for feature, coefficient in zip(X.columns, model.coef_):

    print(feature, ":", round(coefficient, 4))


print("\nIntercept:", round(model.intercept_, 4))


new_house = pd.DataFrame({

    "Appliances": [8],

    "Usage_Hours": [6],

    "House_Size": [1100],

    "Occupants": [4]

})


new_consumption = model.predict(new_house)


print("\nNew House:")

print(new_house)
```

```
print("\nPredicted Consumption:", round(new_consumption[0], 2), "Units")
```

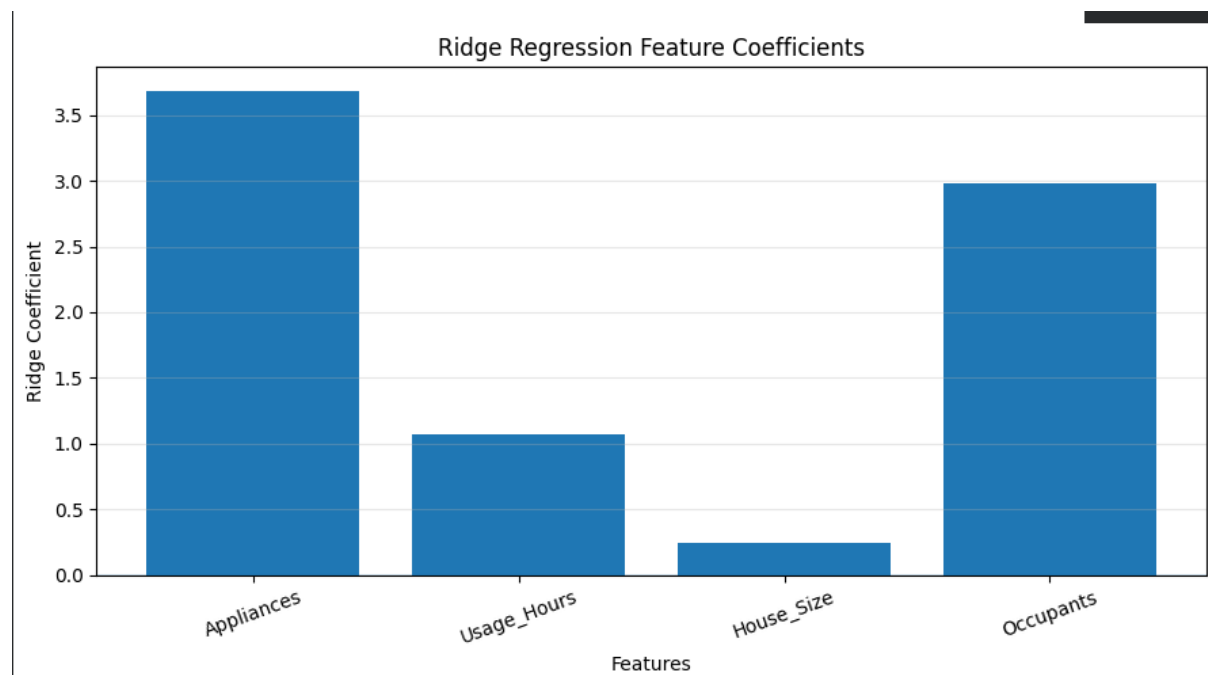
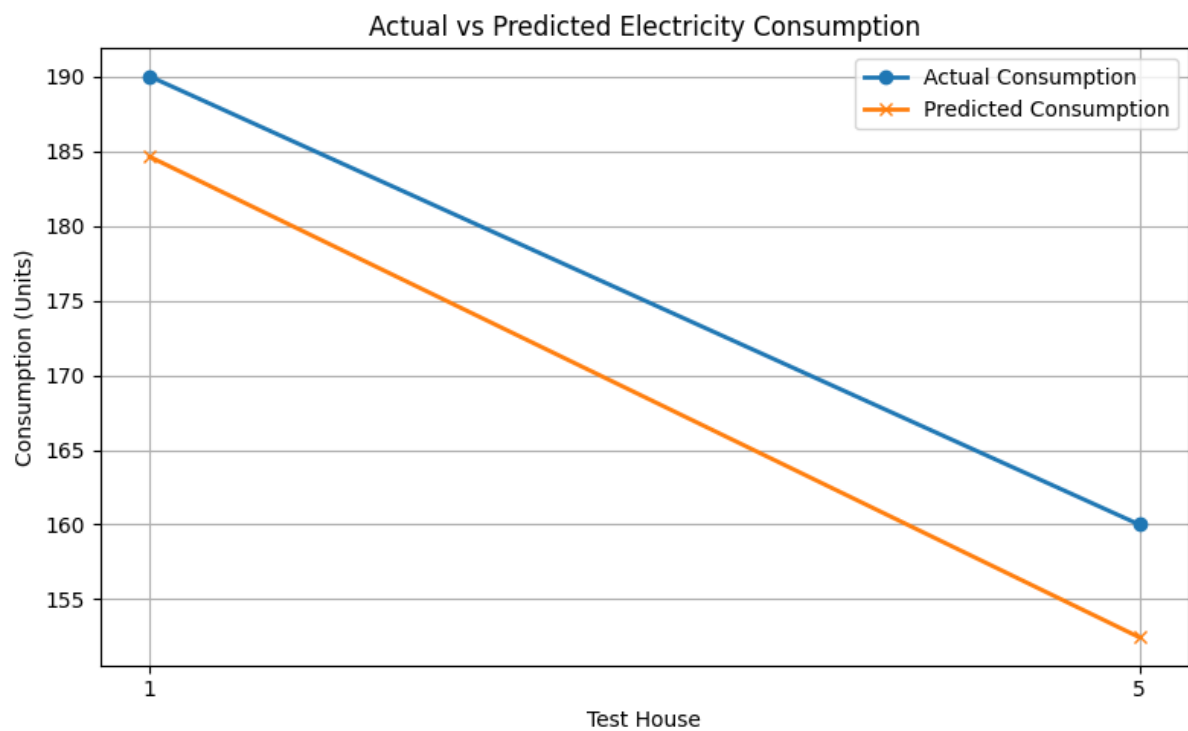
```
plt.figure(figsize=(7, 5))  
plt.scatter(y_test, y_pred, s=80)  
plt.xlabel("Actual Consumption")  
plt.ylabel("Predicted Consumption")  
plt.title("Actual vs Predicted Electricity Consumption")  
plt.grid(True)  
plt.show()
```

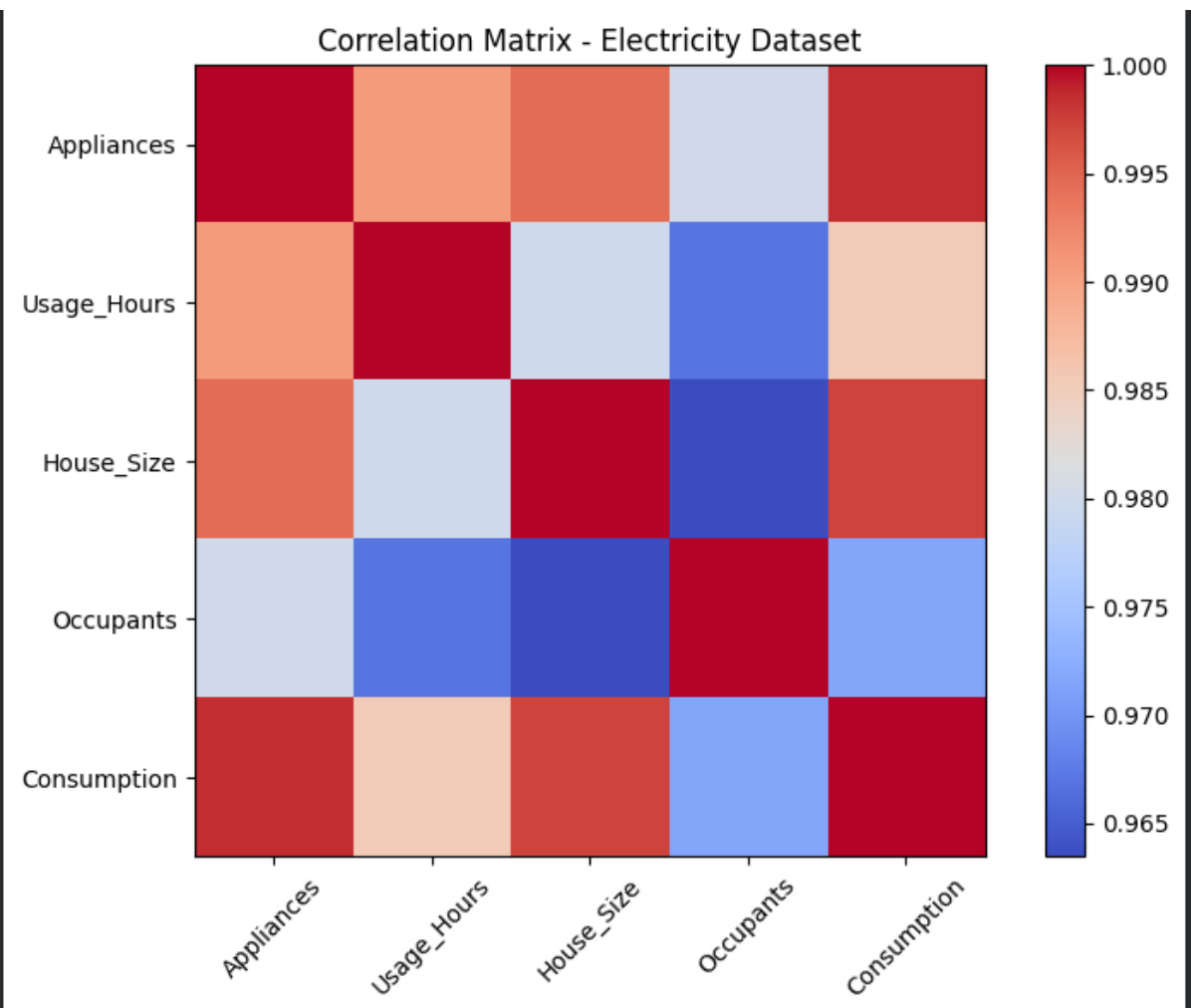
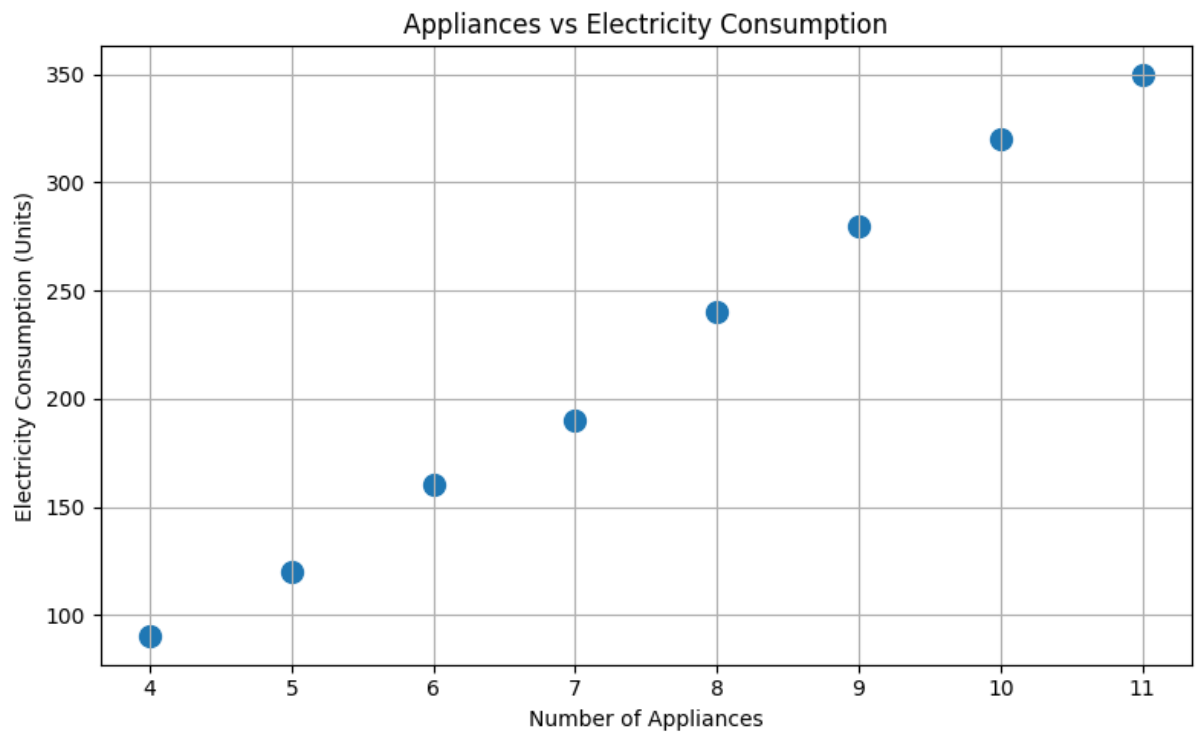
```
plt.figure(figsize=(8, 5))  
plt.bar(X.columns, model.coef_)  
plt.xlabel("Features")  
plt.ylabel("Ridge Coefficient")  
plt.title("Ridge Regression Coefficients")  
plt.xticks(rotation=20)  
plt.grid(axis="y")  
plt.show()
```

```
plt.figure(figsize=(7, 5))  
plt.scatter(df["Usage_Hours"], df["Consumption"], s=80)  
plt.xlabel("Usage Hours")  
plt.ylabel("Consumption")  
plt.title("Usage Hours vs Electricity Consumption")  
plt.grid(True)
```

plt.show()

Output:





Question 5: Delivery Time Prediction Using Linear Regression

A logistics company wants to predict **delivery time** based on distance, traffic level, number of packages, and weather score.

Delivery	Distance km	Traffic Level	Packages	Weather Score	Time Hours
D1	10	2	20	8	1.2
D2	20	4	30	7	2.4
D3	30	6	40	6	3.5
D4	40	8	50	5	4.8
D5	50	9	60	4	6.0
D6	15	3	25	8	1.8
D7	35	7	45	5	4.2
D8	25	5	35	6	3.0

Tasks

- Create the dataset.
- Identify input and output variables.
- Split the data into train and test sets.
- Implement **Linear Regression**.
- Predict delivery time for test records.
- Predict delivery time for:

Distance	Traffic Level	Packages	Weather Score
28	6	38	6

- Evaluate using **MAE, MSE, RMSE, and R² score**.
- Explain how this model can help a logistics company plan delivery schedules.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = {
```

```

"Delivery": ["D1", "D2", "D3", "D4", "D5", "D6", "D7", "D8"],
"Distance_km": [10, 20, 30, 40, 50, 15, 35, 25],
"Traffic_Level": [2, 4, 6, 8, 9, 3, 7, 5],
"Packages": [20, 30, 40, 50, 60, 25, 45, 35],
"Weather_Score": [8, 7, 6, 5, 4, 8, 5, 6],
"Time_Hours": [1.2, 2.4, 3.5, 4.8, 6.0, 1.8, 4.2, 3.0]
}

df = pd.DataFrame(data)

print(df)

X = df[["Distance_km", "Traffic_Level", "Packages", "Weather_Score"]]
y = df["Time_Hours"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nActual Delivery Times:")

```

```
print(y_test.values)
```

```
print("\nPredicted Delivery Times:")
```

```
print(np.round(y_pred, 2))
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("\nModel Evaluation:")
```

```
print("MAE:", round(mae, 4))
```

```
print("MSE:", round(mse, 4))
```

```
print("RMSE:", round(rmse, 4))
```

```
print("R2 Score:", round(r2, 4))
```

```
print("\nRegression Coefficients:")
```

```
for feature, coefficient in zip(X.columns, model.coef_):
```

```
    print(feature, ":", round(coefficient, 4))
```

```
print("\nIntercept:", round(model.intercept_, 4))
```

```
new_delivery = pd.DataFrame({
```

```
    "Distance_km": [28],
```

```
    "Traffic_Level": [6],
```



```
"Packages": [38],  
"Weather_Score": [6]  
})  
  
new_time = model.predict(new_delivery)  
  
print("\nNew Delivery:")  
print(new_delivery)  
  
print("\nPredicted Delivery Time:", round(new_time[0], 2), "Hours")  
  
plt.figure(figsize=(7, 5))  
plt.scatter(y_test, y_pred, s=80)  
plt.xlabel("Actual Delivery Time")  
plt.ylabel("Predicted Delivery Time")  
plt.title("Actual vs Predicted Delivery Time")  
plt.grid(True)  
plt.show()  
  
plt.figure(figsize=(8, 5))  
plt.bar(X.columns, model.coef_)  
plt.xlabel("Features")  
plt.ylabel("Coefficient")  
plt.title("Linear Regression Coefficients")  
plt.xticks(rotation=20)
```

```
plt.grid(axis="y")

plt.show()

plt.figure(figsize=(7, 5))

plt.scatter(df["Distance_km"], df["Time_Hours"], s=80)

plt.xlabel("Distance (km)")

plt.ylabel("Delivery Time (Hours)")

plt.title("Distance vs Delivery Time")

plt.grid(True)

plt.show()
```

Output:

	Delivery	Distance_km	Traffic_Level	Packages	Weather_Score	Time_Hours
0	D1	10	2	20	8	1.2
1	D2	20	4	30	7	2.4
2	D3	30	6	40	6	3.5
3	D4	40	8	50	5	4.8
4	D5	50	9	60	4	6.0
5	D6	15	3	25	8	1.8
6	D7	35	7	45	5	4.2
7	D8	25	5	35	6	3.0

Actual Delivery Times:
[2.4 1.8]

Predicted Delivery Times:
[2.37 1.74]

Model Evaluation:
MAE: 0.0452
MSE: 0.0023
RMSE: 0.048
R2 Score: 0.9744

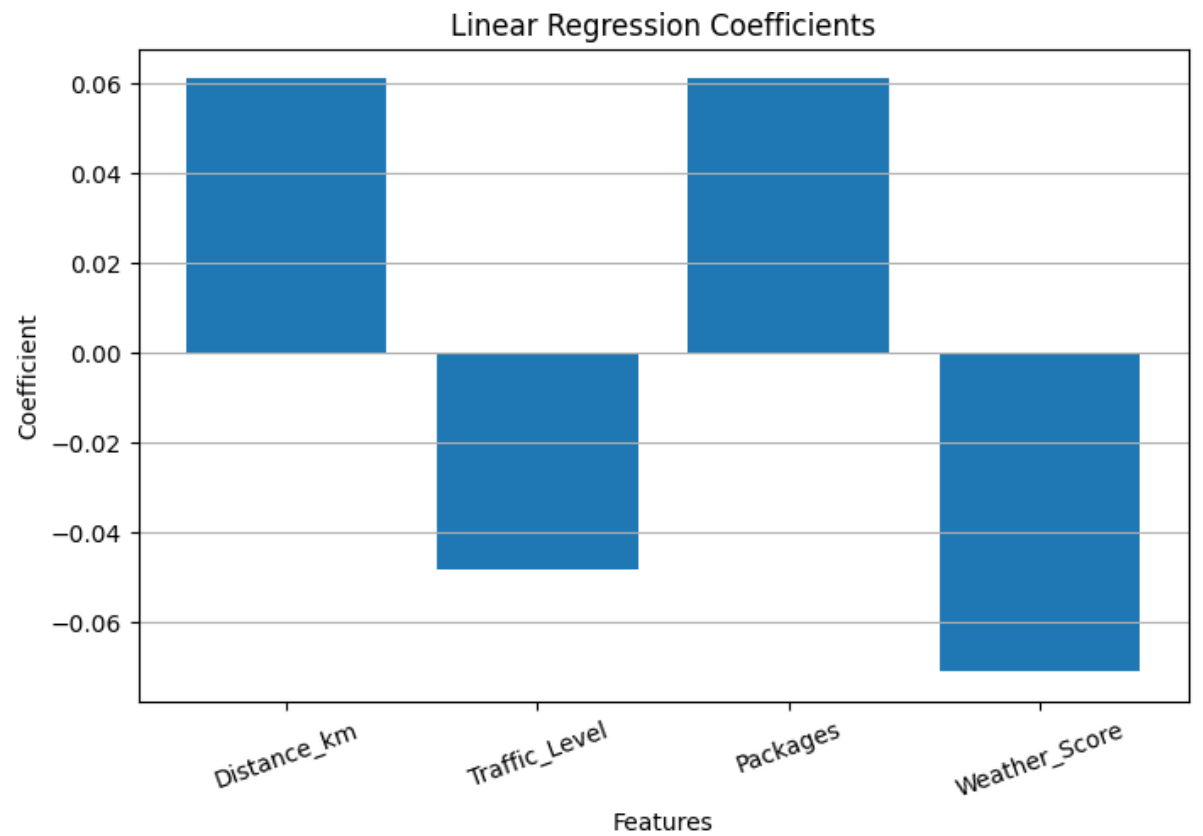
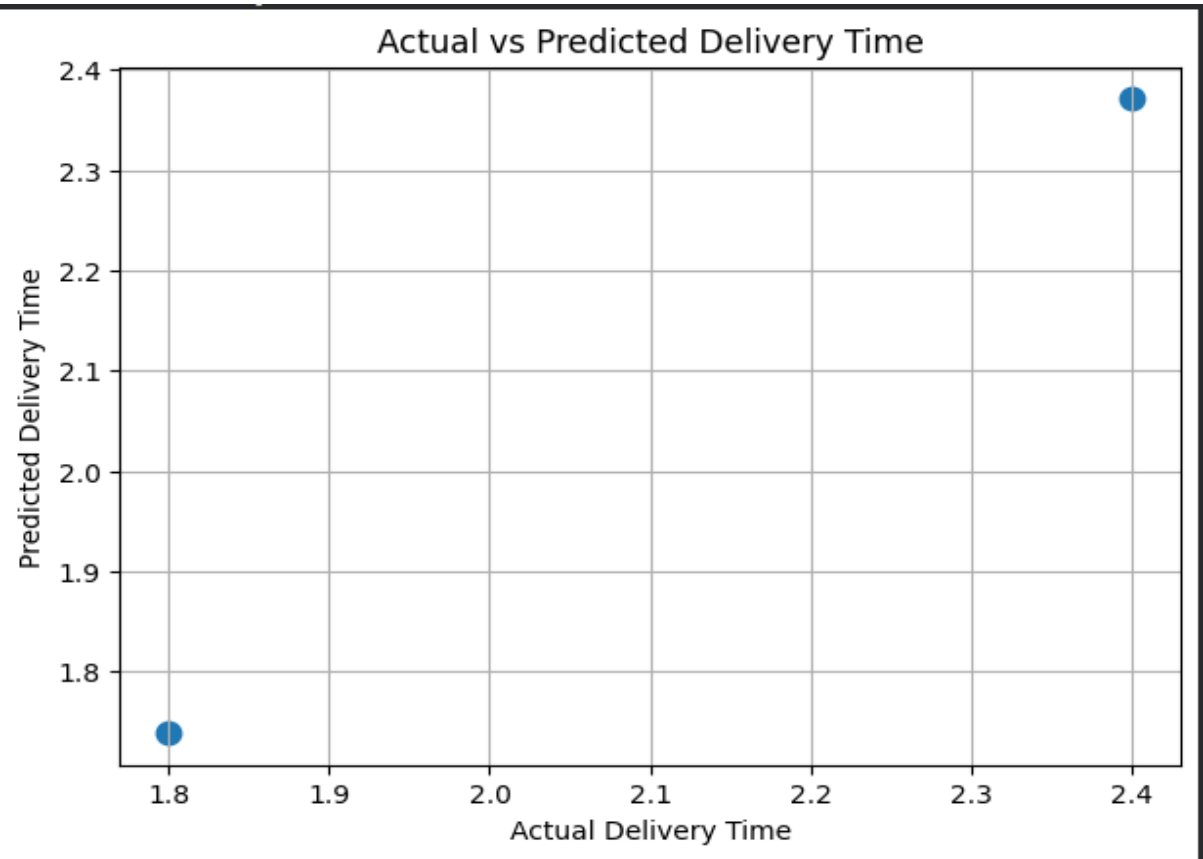
Regression Coefficients:
Distance_km : 0.061
Traffic_Level : -0.0484
Packages : 0.061
Weather_Score : -0.071

Intercept: 0.0129

New Delivery:

	Distance_km	Traffic_Level	Packages	Weather_Score
0	28	6	38	6

Predicted Delivery Time: 3.32 Hours





Question 6: Loan Approval Prediction Using Logistic Regression

A bank wants to predict whether a loan application should be **approved** or **rejected**.

Applicant	Income	Credit Score	Existing Loan	Employment Years	Approved
A1	25000	580	200000	1	0
A2	30000	600	180000	2	0
A3	45000	680	120000	3	1
A4	60000	750	80000	5	1
A5	70000	800	50000	6	1
A6	28000	590	220000	1	0
A7	55000	720	100000	4	1
A8	35000	620	160000	2	0

Here,

Approved = 1 means loan approved.

Approved = 0 means loan rejected.

Tasks

- a) Create the dataset in Python.
- b) Identify independent and dependent variables.
- c) Split the dataset.
- d) Implement **Logistic Regression**.
- e) Predict approval status for test data.
- f) Predict loan approval for a new applicant:

Income	Credit Score	Existing Loan	Employment Years
50000	700	100000	4

- g) Evaluate using **accuracy, confusion matrix, precision, recall, and F1-score**.
- h) Explain why Logistic Regression is suitable for loan approval prediction.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score,
f1_score, ConfusionMatrixDisplay

data = {

    "Applicant": ["A1", "A2", "A3", "A4", "A5", "A6", "A7", "A8"],

    "Income": [25000, 30000, 45000, 60000, 70000, 28000, 55000, 35000],

    "Credit_Score": [580, 600, 680, 750, 800, 590, 720, 620],

    "Existing_Loan": [200000, 180000, 120000, 80000, 50000, 220000, 100000, 160000],

    "Employment_Years": [1, 2, 3, 5, 6, 1, 4, 2],

    "Approved": [0, 0, 1, 1, 1, 0, 1, 0]

}

df = pd.DataFrame(data)
```

```
print(df)
```

```
X = df[["Income", "Credit_Score", "Existing_Loan", "Employment_Years"]]
```

```
y = df["Approved"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42, stratify=y  
)
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
y_prob = model.predict_proba(X_test)[:, 1]
```

```
print("\nActual Approval:")
```

```
print(y_test.values)
```

```
print("\nPredicted Approval:")
```

```
print(y_pred)
```

```
print("\nApproval Probability:")
```

```
print(np.round(y_prob, 2))
```

```
accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred, zero_division=0)

recall = recall_score(y_test, y_pred, zero_division=0)

f1 = f1_score(y_test, y_pred, zero_division=0)

cm = confusion_matrix(y_test, y_pred)
```

```
print("\nModel Evaluation:")

print("Accuracy:", round(accuracy, 4))

print("Precision:", round(precision, 4))

print("Recall:", round(recall, 4))

print("F1 Score:", round(f1, 4))
```

```
print("\nConfusion Matrix:")

print(cm)
```

```
print("\nLogistic Regression Coefficients:")

for feature, coefficient in zip(X.columns, model.coef_[0]):

    print(feature, ":", round(coefficient, 4))
```

```
print("\nIntercept:", round(model.intercept_[0], 4))
```

```
new_applicant = pd.DataFrame({

    "Income": [50000],

    "Credit_Score": [700],

    "Existing_Loan": [100000],
```

```
"Employment_Years": [4]
})

new_prediction = model.predict(new_applicant)
new_probability = model.predict_proba(new_applicant)[0][1]

print("\nNew Applicant:")
print(new_applicant)

print("\nApproval Probability:", round(new_probability * 100, 2), "%")

if new_prediction[0] == 1:
    print("Predicted Decision: Loan Approved")
else:
    print("Predicted Decision: Loan Rejected")

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Rejected", "Approved"]
)

disp.plot(values_format="d")
plt.title("Confusion Matrix - Loan Approval")
plt.show()
```



```

plt.figure(figsize=(8, 5))

plt.bar(X.columns, model.coef_[0])

plt.xlabel("Features")

plt.ylabel("Coefficient")

plt.title("Logistic Regression Coefficients")

plt.xticks(rotation=20)

plt.grid(axis="y")

plt.show()

plt.figure(figsize=(7, 5))

plt.scatter(df["Credit_Score"], df["Approved"], s=80)

plt.xlabel("Credit Score")

plt.ylabel("Approval")

plt.title("Credit Score vs Loan Approval")

plt.yticks([0, 1], ["Rejected", "Approved"])

plt.grid(True)

plt.show()

```

Output:

	Applicant	Income	Credit_Score	Existing_Loan	Employment_Years
0	A1	25000	580	200000	1
0					
1	A2	30000	600	180000	2
0					
2	A3	45000	680	120000	3
1					
3	A4	60000	750	80000	5
1					
4	A5	70000	800	50000	6
1					
5	A6	28000	590	220000	1
0					
6	A7	55000	720	100000	4
1					

7 A8 35000 620 160000 2
0

Actual Approval:
[0 1]

Predicted Approval:
[0 1]

Approval Probability:
[0. 1.]

Model Evaluation:
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

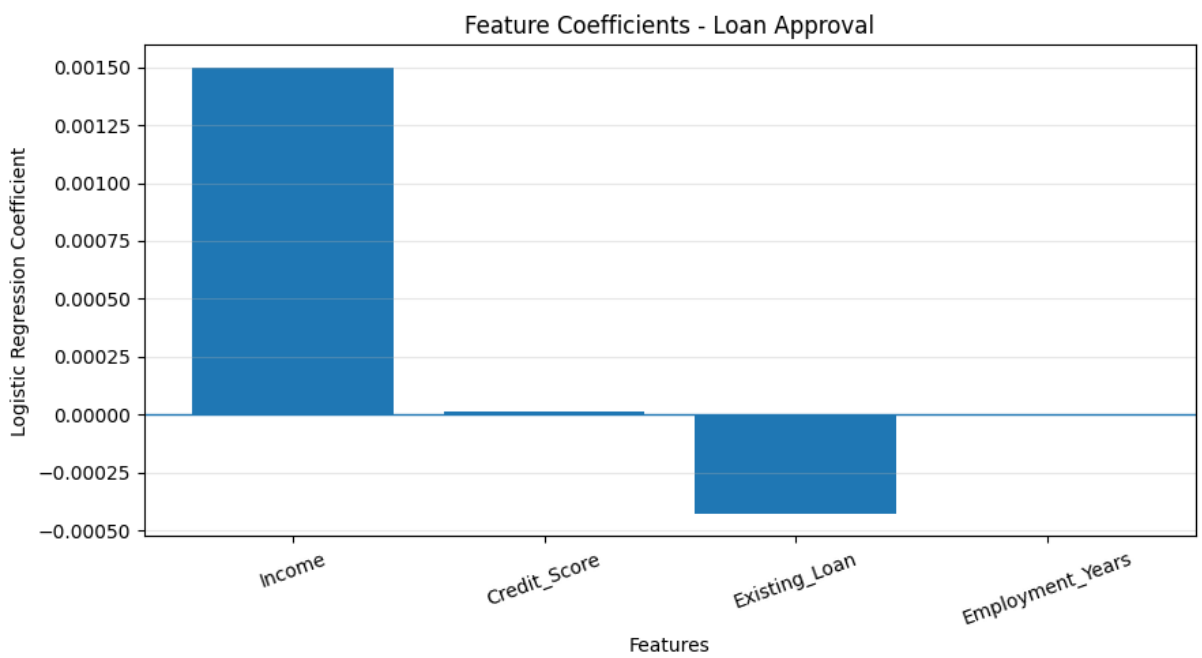
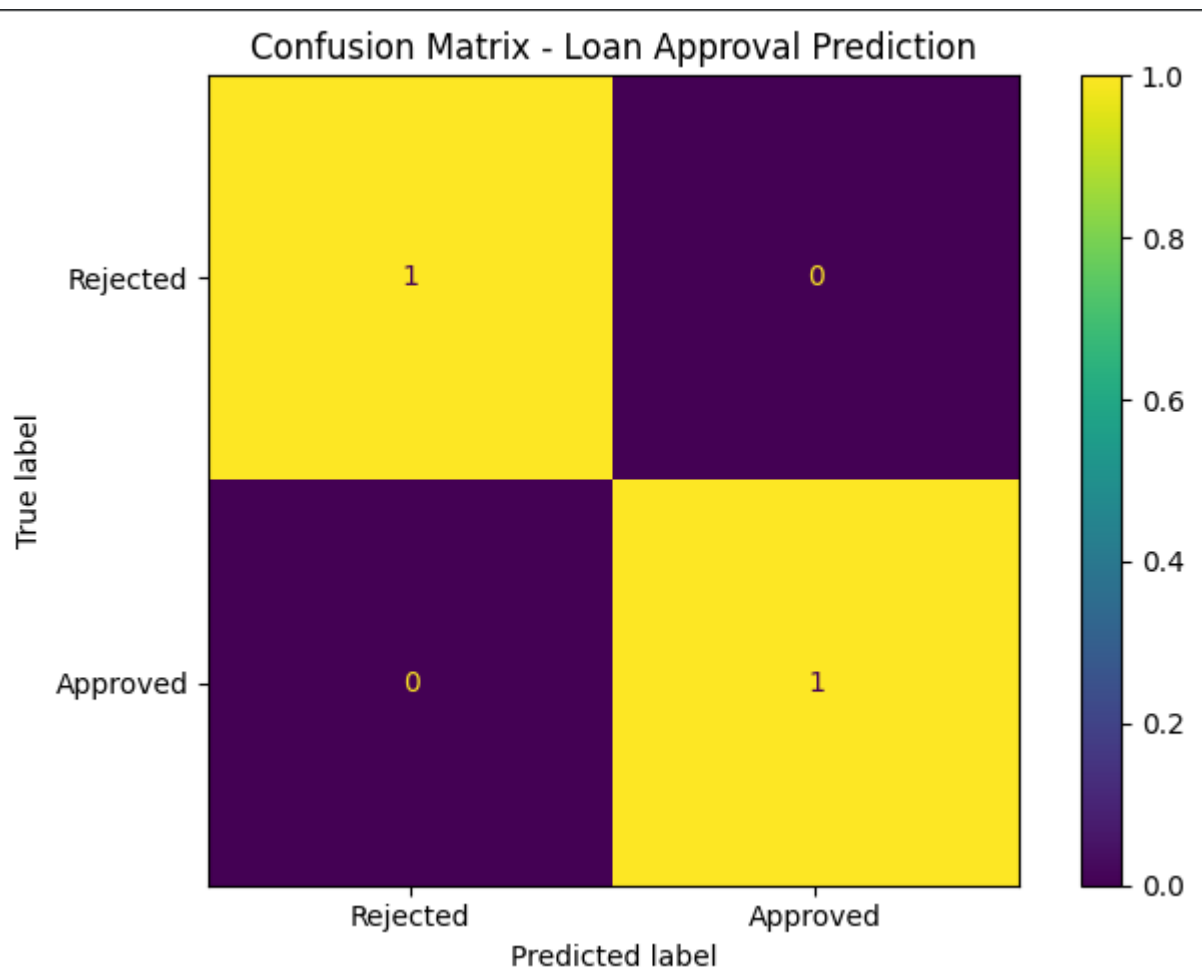
Confusion Matrix:
[[1 0]
[0 1]]

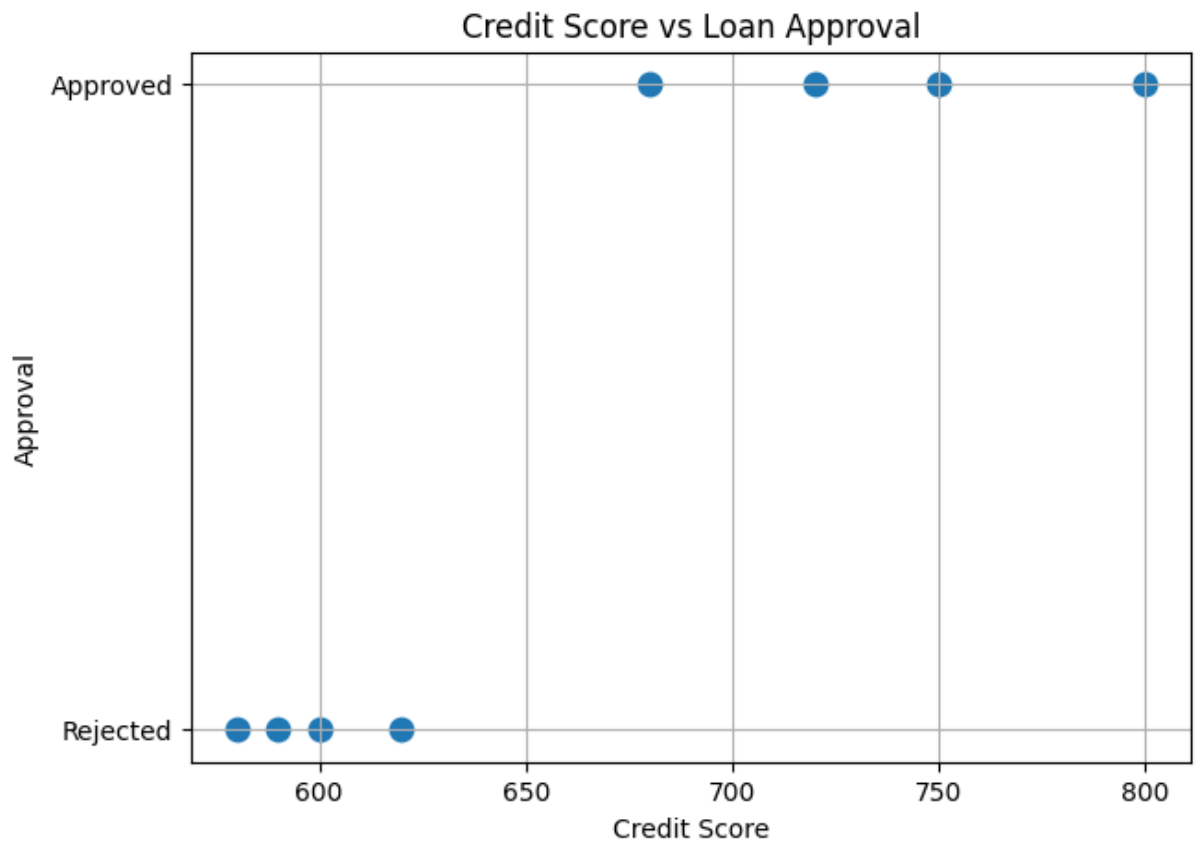
Logistic Regression Coefficients:
Income : 0.0015
Credit_Score : 0.0
Existing_Loan : -0.0004
Employment_Years : 0.0

Intercept: 0.0

New Applicant:
Income Credit_Score Existing_Loan Employment_Years
0 50000 700 100000 4

Approval Probability: 100.0 %
Predicted Decision: Loan Approved





Question 7: Healthcare Treatment Cost Prediction Using Lasso Regression

A hospital wants to predict **treatment cost** based on patient health parameters. Since many medical features are available, the hospital wants to use Lasso Regression to identify important predictors.

Patient	Age	BP	Sugar Level	BMI	Previous Visits	Treatment Cost
P1	25	115	90	22	1	5000
P2	35	125	110	25	2	8000
P3	45	140	150	29	3	15000
P4	55	155	180	32	5	25000
P5	60	165	200	35	6	32000
P6	30	120	100	24	1	7000
P7	50	150	170	31	4	22000
P8	40	135	130	27	2	12000

Tasks

- Create the dataset using Python.
- Identify features and target variable.
- Split into training and testing sets.

- d) Implement **Lasso Regression**.
- e) Try alpha values such as **0.1, 1.0, and 10.0**.
- f) Predict treatment cost for test patients.
- g) Predict treatment cost for a new patient:

Age	BP	Sugar Level	BMI	Previous Visits
48	145	160	30	4

- h) Evaluate using **MAE, MSE, RMSE, and R² score**.
- i) Print the model coefficients.
- j) Explain which medical features are important based on Lasso coefficients.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Lasso

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

data = {

    "Patient": ["P1", "P2", "P3", "P4", "P5", "P6", "P7", "P8"],

    "Age": [25, 35, 45, 55, 60, 30, 50, 40],

    "BP": [115, 125, 140, 155, 165, 120, 150, 135],

    "Sugar_Level": [90, 110, 150, 180, 200, 100, 170, 130],

    "BMI": [22, 25, 29, 32, 35, 24, 31, 27],

    "Previous_Visits": [1, 2, 3, 5, 6, 1, 4, 2],

    "Treatment_Cost": [5000, 8000, 15000, 25000, 32000, 7000, 22000, 12000]

}

df = pd.DataFrame(data)
```

```
print(df)
```

```
X = df[["Age", "BP", "Sugar_Level", "BMI", "Previous_Visits"]]
```

```
y = df["Treatment_Cost"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.25, random_state=42
```

```
)
```

```
model = Lasso(alpha=0.1, max_iter=10000)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("\nActual Treatment Costs:")
```

```
print(y_test.values)
```

```
print("\nPredicted Treatment Costs:")
```

```
print(np.round(y_pred, 2))
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("\nModel Evaluation:")

print("MAE:", round(mae, 4))

print("MSE:", round(mse, 4))

print("RMSE:", round(rmse, 4))

print("R2 Score:", round(r2, 4))


print("\nLasso Coefficients:")

for feature, coefficient in zip(X.columns, model.coef_):

    print(feature, ":", round(coefficient, 4))


print("\nIntercept:", round(model.intercept_, 4))


new_patient = pd.DataFrame({

    "Age": [48],

    "BP": [145],

    "Sugar_Level": [160],

    "BMI": [30],

    "Previous_Visits": [4]

})


new_cost = model.predict(new_patient)


print("\nNew Patient:")

print(new_patient)
```

```
print("\nPredicted Treatment Cost:", round(new_cost[0], 2))
```

```
plt.figure(figsize=(7, 5))
```

```
plt.scatter(y_test, y_pred, s=80)
```

```
plt.xlabel("Actual Treatment Cost")
```

```
plt.ylabel("Predicted Treatment Cost")
```

```
plt.title("Actual vs Predicted Treatment Cost")
```

```
plt.grid(True)
```

```
plt.show()
```

```
plt.figure(figsize=(8, 5))
```

```
plt.bar(X.columns, model.coef_)
```

```
plt.xlabel("Medical Features")
```

```
plt.ylabel("Lasso Coefficient")
```

```
plt.title("Lasso Regression Coefficients")
```

```
plt.xticks(rotation=20)
```

```
plt.grid(axis="y")
```

```
plt.show()
```

```
plt.figure(figsize=(7, 5))
```

```
plt.scatter(df["Age"], df["Treatment_Cost"], s=80)
```

```
plt.xlabel("Age")
```

```
plt.ylabel("Treatment Cost")
```

```
plt.title("Age vs Treatment Cost")
```



```
plt.grid(True)
```

```
plt.show()
```

```
alphas = [0.1, 1.0, 10.0]
```

```
results = []
```

```
for alpha in alphas:
```

```
    lasso = Lasso(alpha=alpha, max_iter=10000)
```

```
    lasso.fit(X_train, y_train)
```

```
    prediction = lasso.predict(X_test)
```

```
    results.append([
```

```
        alpha,
```

```
        mean_absolute_error(y_test, prediction),
```

```
        mean_squared_error(y_test, prediction),
```

```
        np.sqrt(mean_squared_error(y_test, prediction)),
```

```
        r2_score(y_test, prediction)
```

```
    ])
```

```
alpha_df = pd.DataFrame(
```

```
    results,
```

```
    columns=["Alpha", "MAE", "MSE", "RMSE", "R2"]
```

```
)
```

```
print("\nAlpha Comparison:")
```

```
print(alpha_df)
```

```
plt.figure(figsize=(7, 5))
```

```
plt.plot(alpha_df["Alpha"], alpha_df["R2"], marker="o")
```

```
plt.xlabel("Alpha")
```

```
plt.ylabel("R2 Score")
```

```
plt.title("Alpha vs R2 Score")
```

```
plt.grid(True)
```

```
plt.show()
```

Output:

	Patient	Age	BP	Sugar_Level	BMI	Previous_Visits	Treatment_Cost
0	P1	25	115	90	22	1	5000
1	P2	35	125	110	25	2	8000
2	P3	45	140	150	29	3	15000
3	P4	55	155	180	32	5	25000
4	P5	60	165	200	35	6	32000
5	P6	30	120	100	24	1	7000
6	P7	50	150	170	31	4	22000
7	P8	40	135	130	27	2	12000

```
Actual Treatment Costs:  
[8000 7000]
```

```
Predicted Treatment Costs:  
[8170.43 5040.51]
```

```
Model Evaluation:
```

```
MAE: 1064.9595
```

```
MSE: 1934326.148
```

```
RMSE: 1390.8005
```

```
R2 Score: -6.7373
```

```
Lasso Coefficients:
```

```
Age : -753.4188
```

```
BP : 816.3621
```

```
Sugar_Level : -23.2589
```

```
BMI : -13.7054
```

```
Previous_Visits : 3061.4964
```

```
Intercept: -70727.0615
```

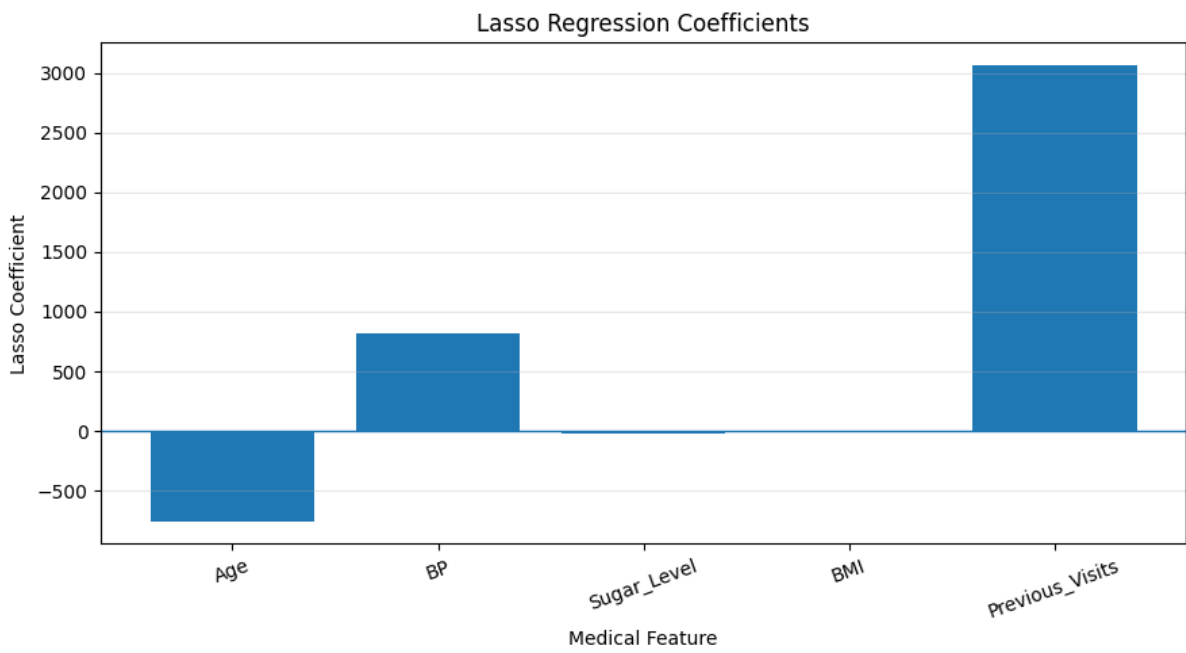
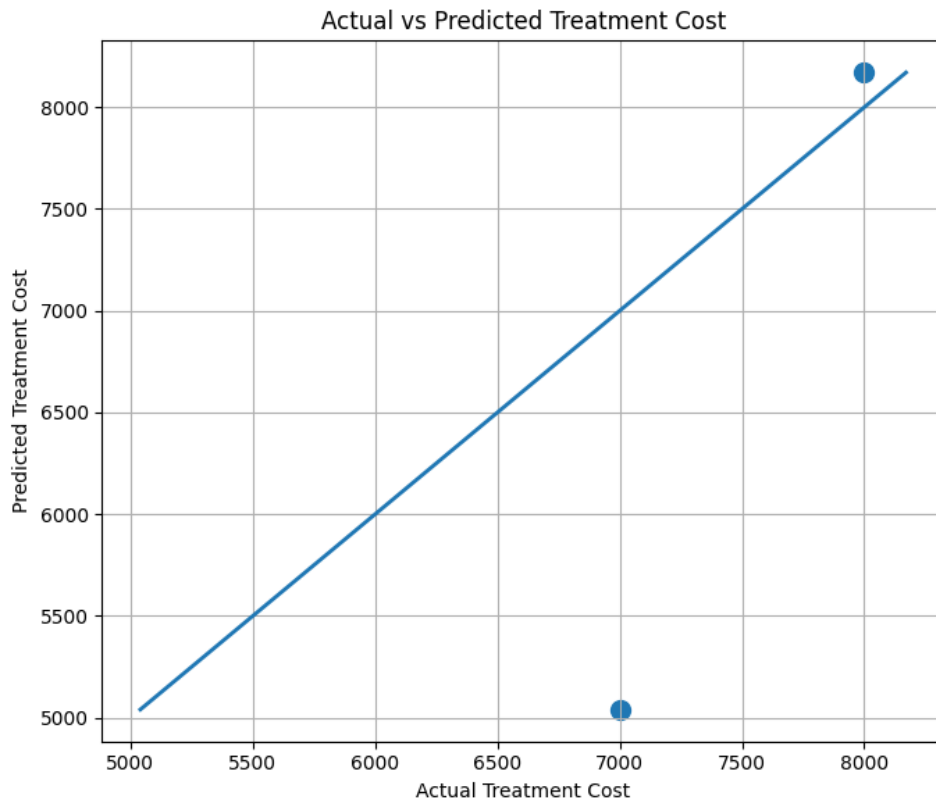
```
New Patient:
```

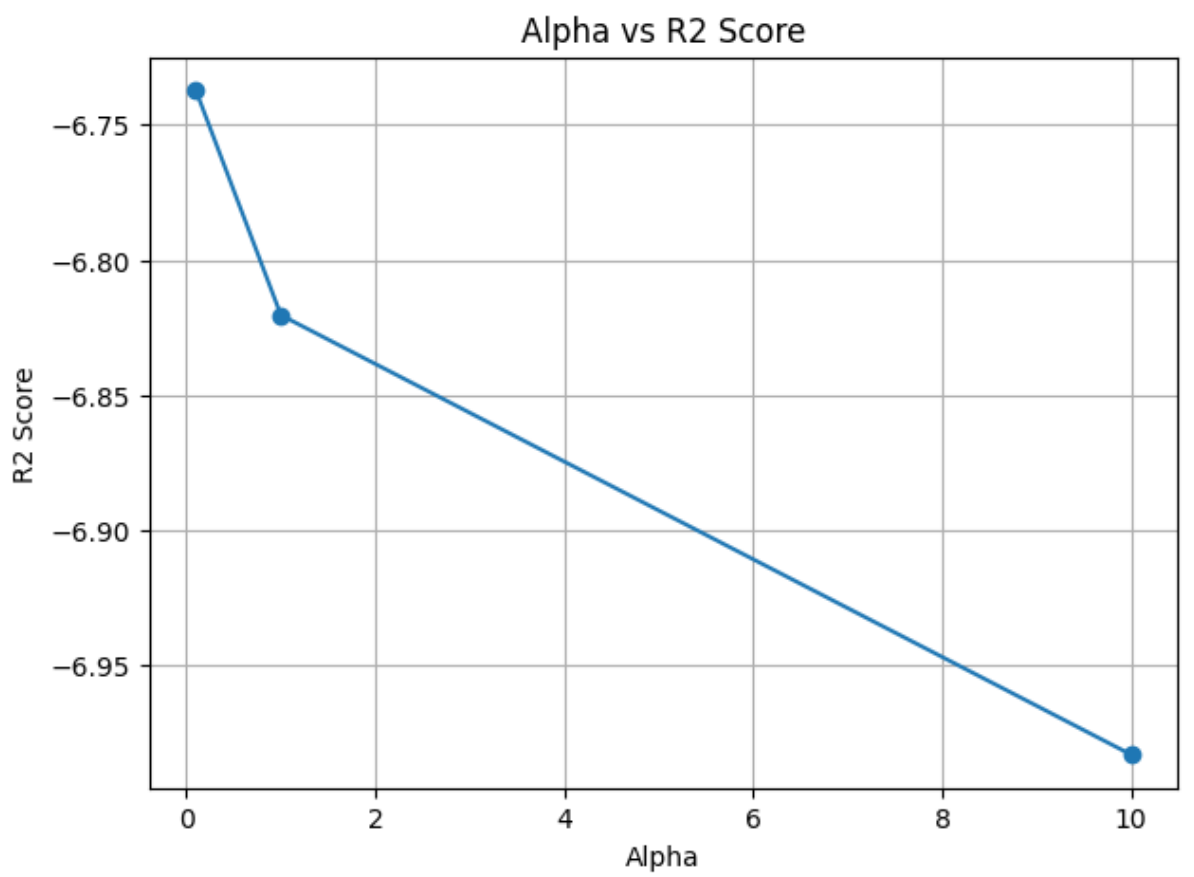
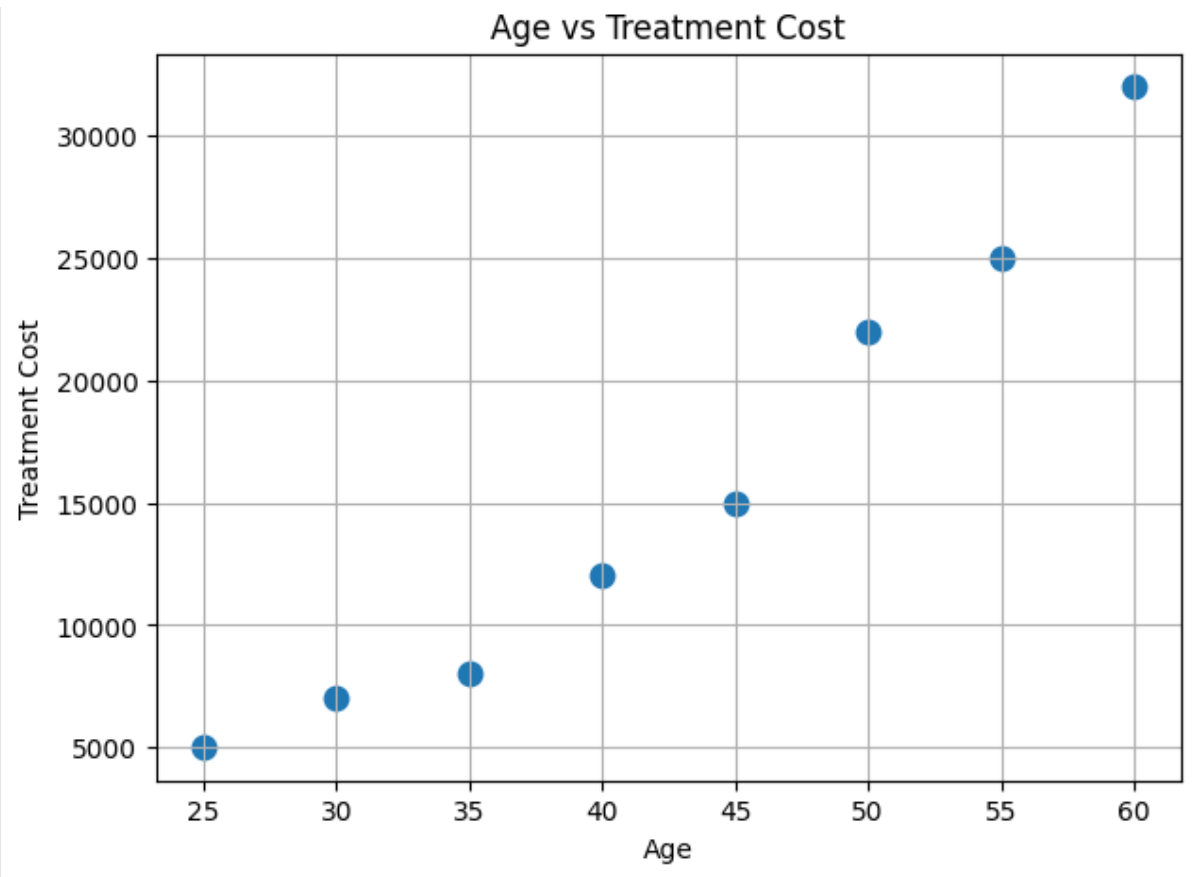
	Age	BP	Sugar_Level	BMI	Previous_Visits
0	48	145	160	30	4

```
Predicted Treatment Cost: 19594.75
```

Alpha Comparison:

	Alpha	MAE	MSE	RMSE	R2
0	0.1	1064.959520	1.934326e+06	1390.800542	-6.737305
1	1.0	1058.597336	1.955101e+06	1398.249358	-6.820405
2	10.0	1017.986033	1.995729e+06	1412.702643	-6.982915





Question 8: Manufacturing Defect Strength Prediction Using Ridge Regression

A manufacturing company wants to predict the **strength score** of a product based on machine parameters. Since temperature, pressure, and machine speed may be correlated, Ridge Regression is used.

Product	Temperature	Pressure	Machine Speed	Material Quality	Strength Score
P1	70	30	100	8	75
P2	75	35	110	7	78
P3	80	38	120	6	80
P4	85	42	125	6	82
P5	90	45	130	5	79
P6	95	50	140	4	76
P7	78	36	115	7	81
P8	88	44	128	5	80

Tasks

- Create the dataset.
- Identify the independent variables and dependent variable.
- Split into training and testing data.
- Implement **Ridge Regression**.
- Use alpha values such as **0.1, 1.0, and 10.0**.
- Predict strength score for test products.
- Predict strength score for a new product:

Temperature	Pressure	Machine Speed	Material Quality
82	40	122	6

- Evaluate using **MAE, MSE, RMSE, and R² score**.
- Display Ridge coefficients.
- Explain how Ridge handles multicollinearity in manufacturing data.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import Ridge
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
data = {  
    "Product": ["P1", "P2", "P3", "P4", "P5", "P6", "P7", "P8"],  
    "Temperature": [70, 75, 80, 85, 90, 95, 78, 88],  
    "Pressure": [30, 35, 38, 42, 45, 50, 36, 44],  
    "Machine_Speed": [100, 110, 120, 125, 130, 140, 115, 128],  
    "Material_Quality": [8, 7, 6, 6, 5, 4, 7, 5],  
    "Strength_Score": [75, 78, 80, 82, 79, 76, 81, 80]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

```
X = df[["Temperature", "Pressure", "Machine_Speed", "Material_Quality"]]
```

```
y = df["Strength_Score"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42
```

```
)
```

```
model = Ridge(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("\nActual Strength Scores:")

print(y_test.values)


print("\nPredicted Strength Scores:")

print(np.round(y_pred, 2))


mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)


print("\nModel Evaluation:")

print("MAE:", round(mae, 4))

print("MSE:", round(mse, 4))

print("RMSE:", round(rmse, 4))

print("R2 Score:", round(r2, 4))


print("\nRidge Coefficients:")

for feature, coefficient in zip(X.columns, model.coef_):

    print(feature, ":", round(coefficient, 4))


print("\nIntercept:", round(model.intercept_, 4))


new_product = pd.DataFrame({

    "Temperature": [82],
```



```

    "Pressure": [40],
    "Machine_Speed": [122],
    "Material_Quality": [6]
})

new_strength = model.predict(new_product)

print("\nNew Product:")
print(new_product)

print("\nPredicted Strength Score:", round(new_strength[0], 2))

plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred, s=80)
plt.xlabel("Actual Strength Score")
plt.ylabel("Predicted Strength Score")
plt.title("Actual vs Predicted Strength Score")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.bar(X.columns, model.coef_)
plt.xlabel("Manufacturing Features")
plt.ylabel("Ridge Coefficient")
plt.title("Ridge Regression Coefficients")

```

```
plt.xticks(rotation=20)
```

```
plt.grid(axis="y")
```

```
plt.show()
```

```
plt.figure(figsize=(7, 5))
```

```
plt.scatter(df["Temperature"], df["Strength_Score"], s=80)
```

```
plt.xlabel("Temperature")
```

```
plt.ylabel("Strength Score")
```

```
plt.title("Temperature vs Strength Score")
```

```
plt.grid(True)
```

```
plt.show()
```

```
alphas = [0.1, 1.0, 10.0]
```

```
results = []
```

```
for alpha in alphas:
```

```
    ridge = Ridge(alpha=alpha)
```

```
    ridge.fit(X_train, y_train)
```

```
    prediction = ridge.predict(X_test)
```

```
    results.append([
```

```
        alpha,
```

```
        mean_absolute_error(y_test, prediction),
```

```
        mean_squared_error(y_test, prediction),
```

```
        np.sqrt(mean_squared_error(y_test, prediction)),
```

```

        r2_score(y_test, prediction)

    ])

alpha_df = pd.DataFrame(

    results,

    columns=["Alpha", "MAE", "MSE", "RMSE", "R2"]

)

print("\nAlpha Comparison:")

print(alpha_df)

plt.figure(figsize=(7, 5))

plt.plot(alpha_df["Alpha"], alpha_df["R2"], marker="o")

plt.xlabel("Alpha")

plt.ylabel("R2 Score")

plt.title("Alpha vs R2 Score")

plt.grid(True)

plt.show()

```

Output:

	Product	Temperature	Pressure	Machine_Speed	Material_Quality	\
0	P1	70	30	100	8	
1	P2	75	35	110	7	
2	P3	80	38	120	6	
3	P4	85	42	125	6	
4	P5	90	45	130	5	
5	P6	95	50	140	4	
6	P7	78	36	115	7	
7	P8	88	44	128	5	

	Strength_Score
0	75
1	78
2	80

3	82
4	79
5	76
6	81
7	80

Actual Strength Scores:
[78 76]

Predicted Strength Scores:
[78.11 82.56]

Model Evaluation:
MAE: 3.3354
MSE: 21.5103
RMSE: 4.6379
R2 Score: -20.5103

Ridge Coefficients:
Temperature : -0.6724
Pressure : -0.1134
Machine_Speed : 0.7587
Material_Quality : 1.0556

Intercept: 41.665

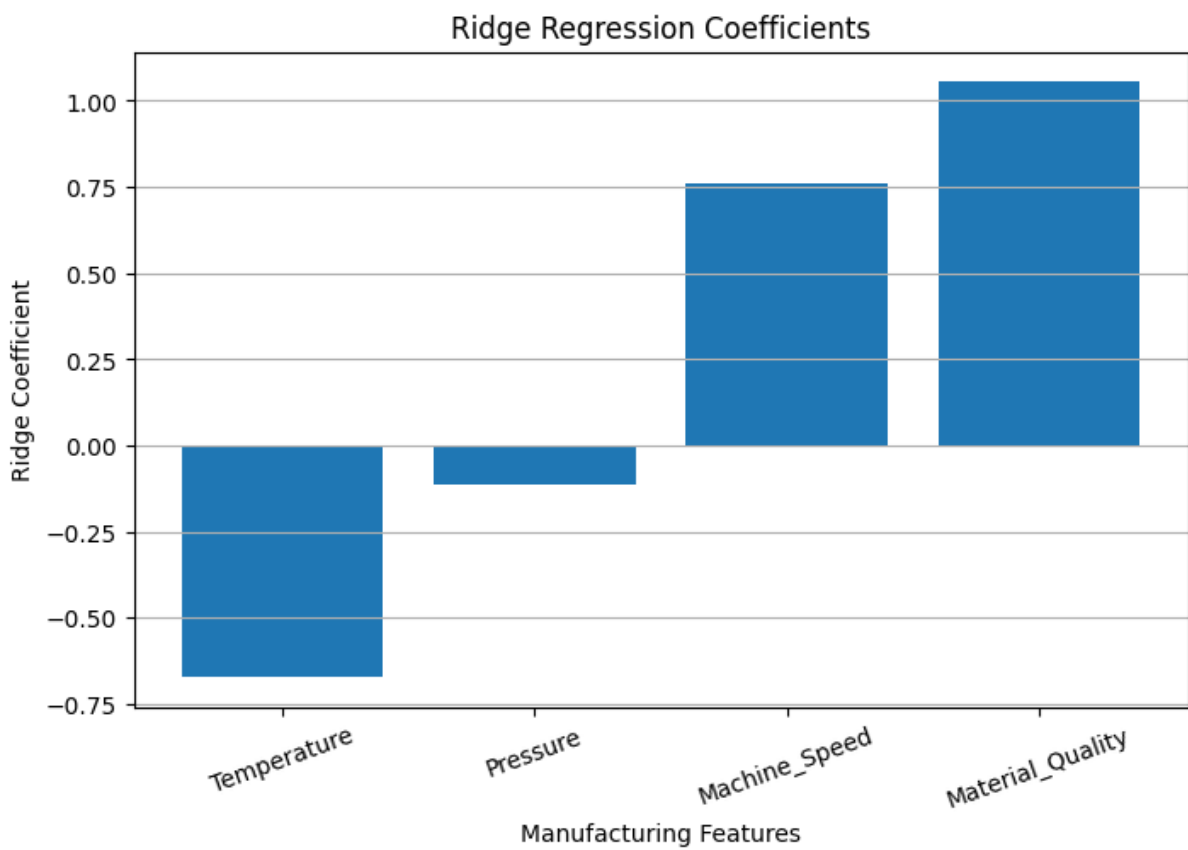
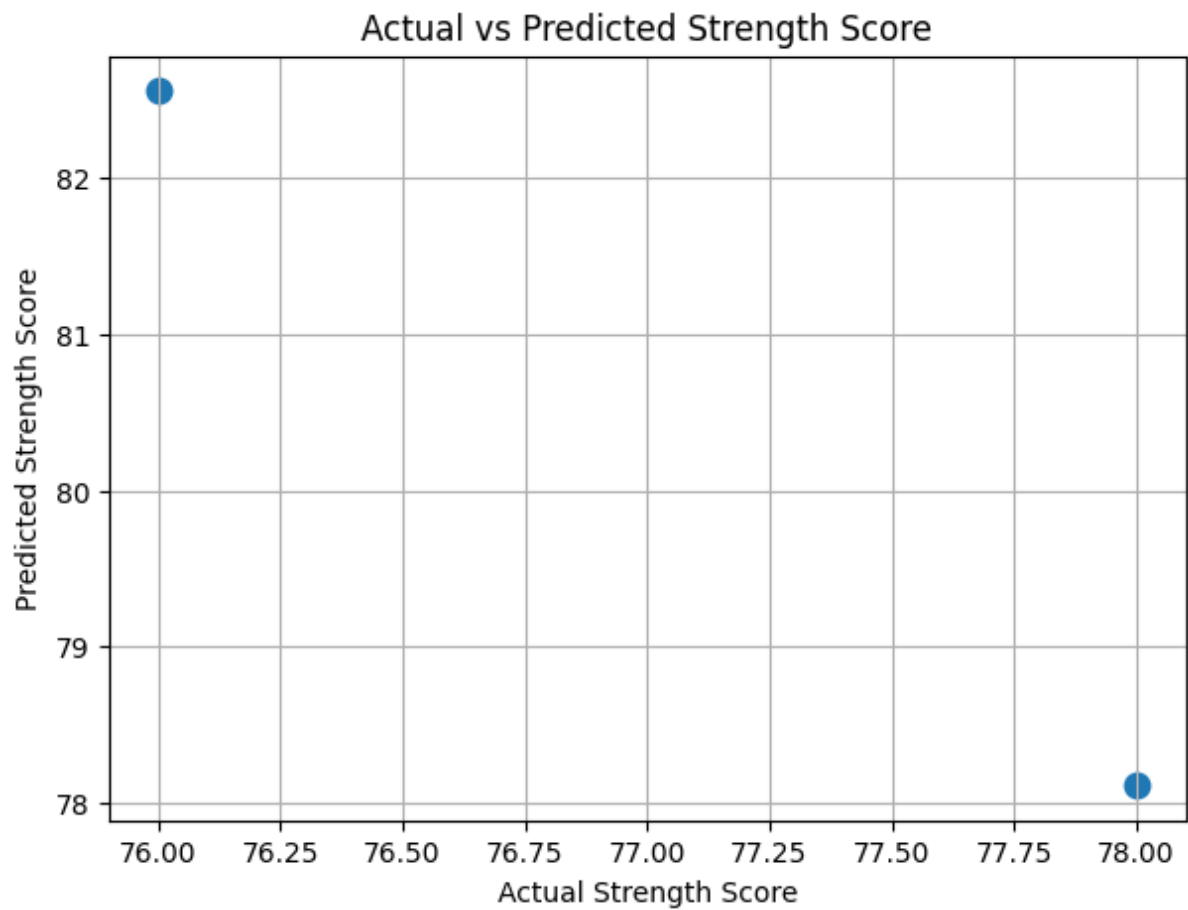
New Product:

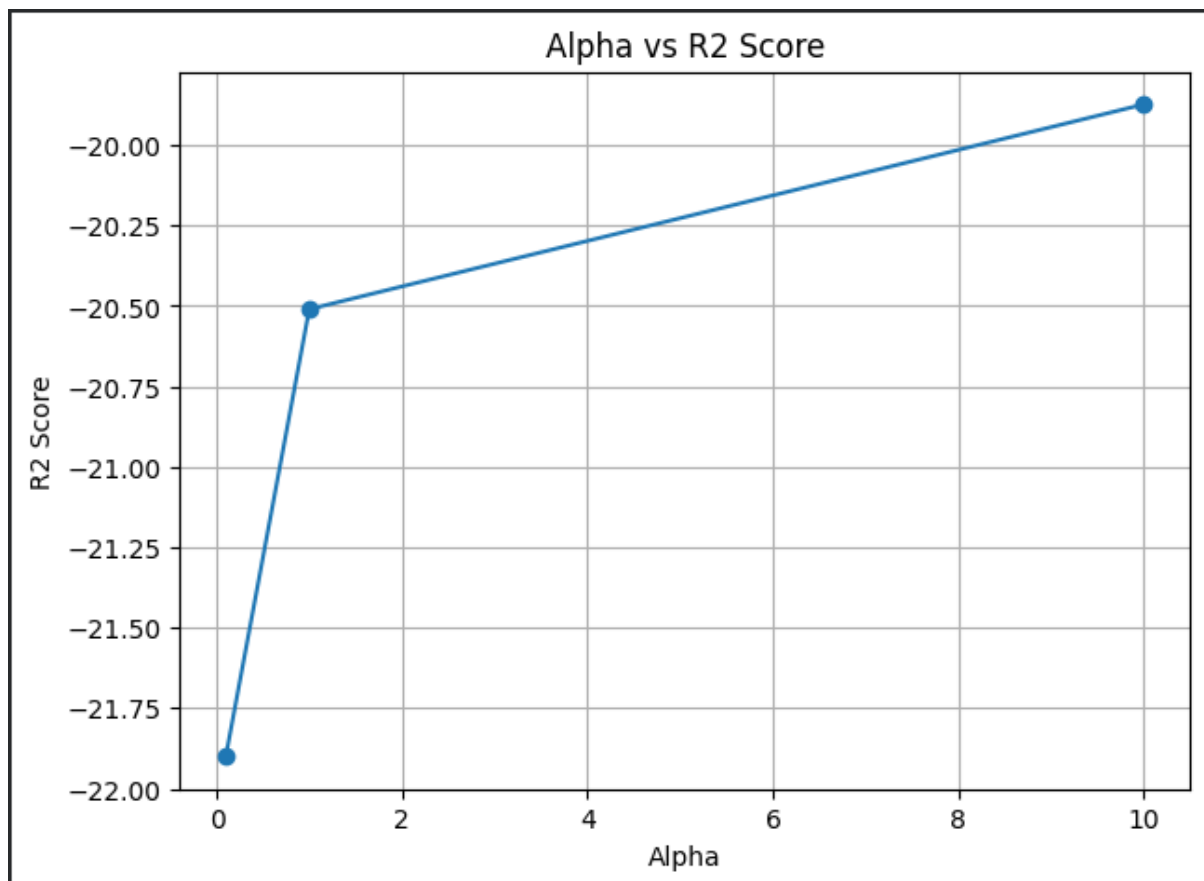
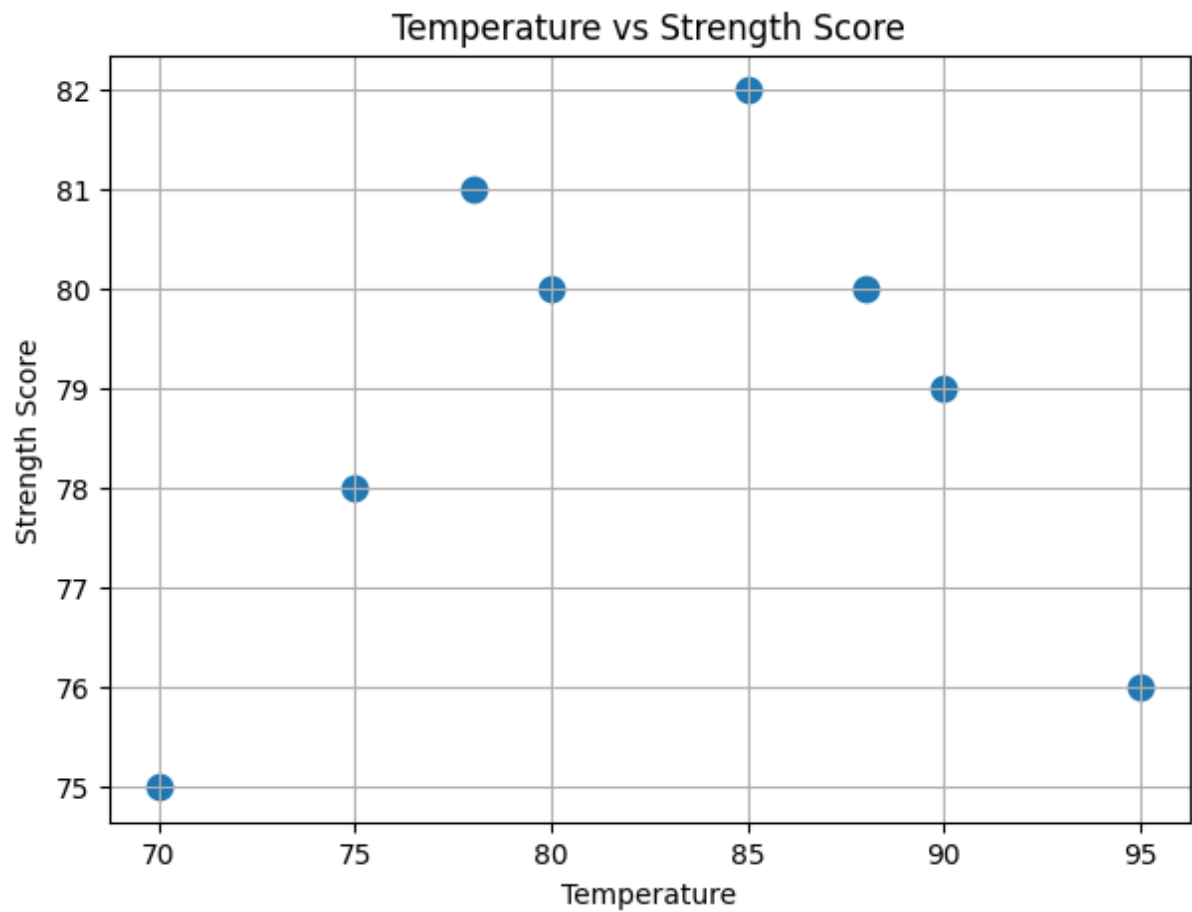
	Temperature	Pressure	Machine_Speed	Material_Quality
0	82	40	122	6

Predicted Strength Score: 80.89

Alpha Comparison:

	Alpha	MAE	MSE	RMSE	R2
0	0.1	3.551200	22.899934	4.785388	-21.899934
1	1.0	3.335405	21.510284	4.637918	-20.510284
2	10.0	3.283425	20.873756	4.568781	-19.873756





Question 9: Student Salary Package Prediction Using Linear Regression

A university placement cell wants to predict the **expected salary package** of students.

Student	CGPA	Aptitude Score	Coding Score	Communication Score	Package LPA
S1	6.5	60	55	65	3.2
S2	7.0	65	60	70	4.0
S3	7.5	70	68	72	5.0
S4	8.0	78	80	75	6.5
S5	8.5	85	88	80	8.0
S6	9.0	90	92	85	10.0
S7	6.8	62	58	68	3.8
S8	8.2	80	82	78	7.0

Tasks

- Create the dataset using Python.
- Identify input and output variables.
- Split the dataset.
- Implement **Linear Regression**.
- Predict package for test students.
- Predict package for a new student:

CGPA	Aptitude Score	Coding Score	Communication Score
8.1	82	85	78

- Evaluate using **MAE, MSE, RMSE, and R² score**.
- Interpret the result from the placement cell perspective.

Code:

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
data = {
```

```
    "Student": ["S1", "S2", "S3", "S4", "S5", "S6", "S7", "S8"],
```

```

"CGPA": [6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 6.8, 8.2],
"Aptitude_Score": [60, 65, 70, 78, 85, 90, 62, 80],
"Coding_Score": [55, 60, 68, 80, 88, 92, 58, 82],
"Communication_Score": [65, 70, 72, 75, 80, 85, 68, 78],
"Package_LPA": [3.2, 4.0, 5.0, 6.5, 8.0, 10.0, 3.8, 7.0]
}

df = pd.DataFrame(data)

print(df)

X = df[["CGPA", "Aptitude_Score", "Coding_Score", "Communication_Score"]]
y = df["Package_LPA"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nActual Packages:")
print(y_test.values)

```



```
print("\nPredicted Packages:")

print(np.round(y_pred, 2))


mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)


print("\nModel Evaluation:")

print("MAE:", round(mae, 4))

print("MSE:", round(mse, 4))

print("RMSE:", round(rmse, 4))

print("R2 Score:", round(r2, 4))


print("\nRegression Coefficients:")

for feature, coefficient in zip(X.columns, model.coef_):

    print(feature, ":", round(coefficient, 4))


print("\nIntercept:", round(model.intercept_, 4))


new_student = pd.DataFrame({

    "CGPA": [8.1],

    "Aptitude_Score": [82],

    "Coding_Score": [85],
```

```
"Communication_Score": [78]
})

new_package = model.predict(new_student)

print("\nNew Student:")
print(new_student)

print("\nPredicted Salary Package:", round(new_package[0], 2), "LPA")

plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred, s=80)
plt.xlabel("Actual Package (LPA)")
plt.ylabel("Predicted Package (LPA)")
plt.title("Actual vs Predicted Salary Package")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.bar(X.columns, model.coef_)
plt.xlabel("Features")
plt.ylabel("Coefficient")
plt.title("Linear Regression Coefficients")
plt.xticks(rotation=20)
plt.grid(axis="y")
```

```
plt.show()

plt.figure(figsize=(7, 5))

plt.scatter(df["CGPA"], df["Package_LPA"], s=80)

plt.xlabel("CGPA")

plt.ylabel("Package (LPA)")

plt.title("CGPA vs Salary Package")

plt.grid(True)

plt.show()
```

Output:

	Student	CGPA	Aptitude_Score	Coding_Score	Communication_Score
0	S1	6.5	60	55	65
1	S2	7.0	65	60	70
2	S3	7.5	70	68	72
3	S4	8.0	78	80	75
4	S5	8.5	85	88	80
5	S6	9.0	90	92	85
6	S7	6.8	62	58	68
7	S8	8.2	80	82	78

	Package_LPA
0	3.2
1	4.0
2	5.0
3	6.5
4	8.0
5	10.0
6	3.8
7	7.0

Actual Packages:
[4. 10.]

Predicted Packages:
[4.3 8.96]

Model Evaluation:
MAE: 0.667
MSE: 0.5812
RMSE: 0.7623
R2 Score: 0.9354

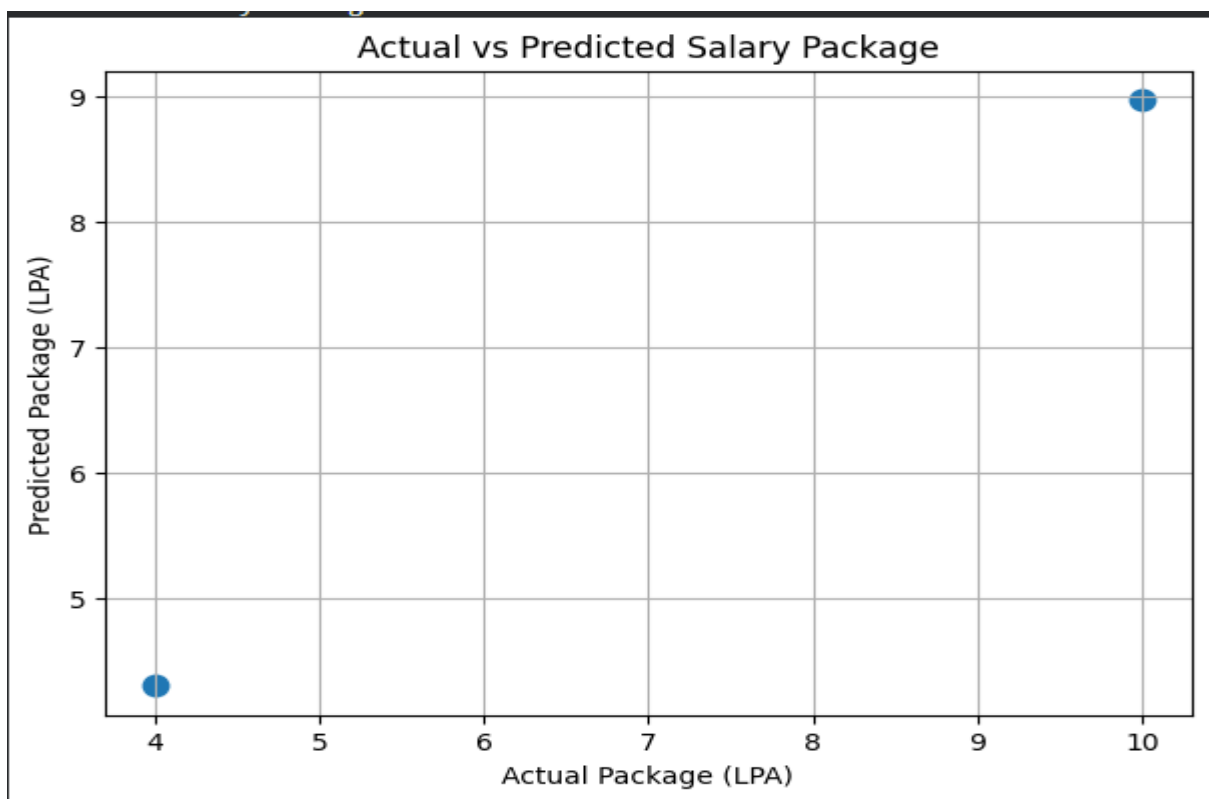
Regression Coefficients:
CGPA : -1.3917
Aptitude_Score : 0.1275
Coding_Score : 0.0532
Communication_Score : 0.1708

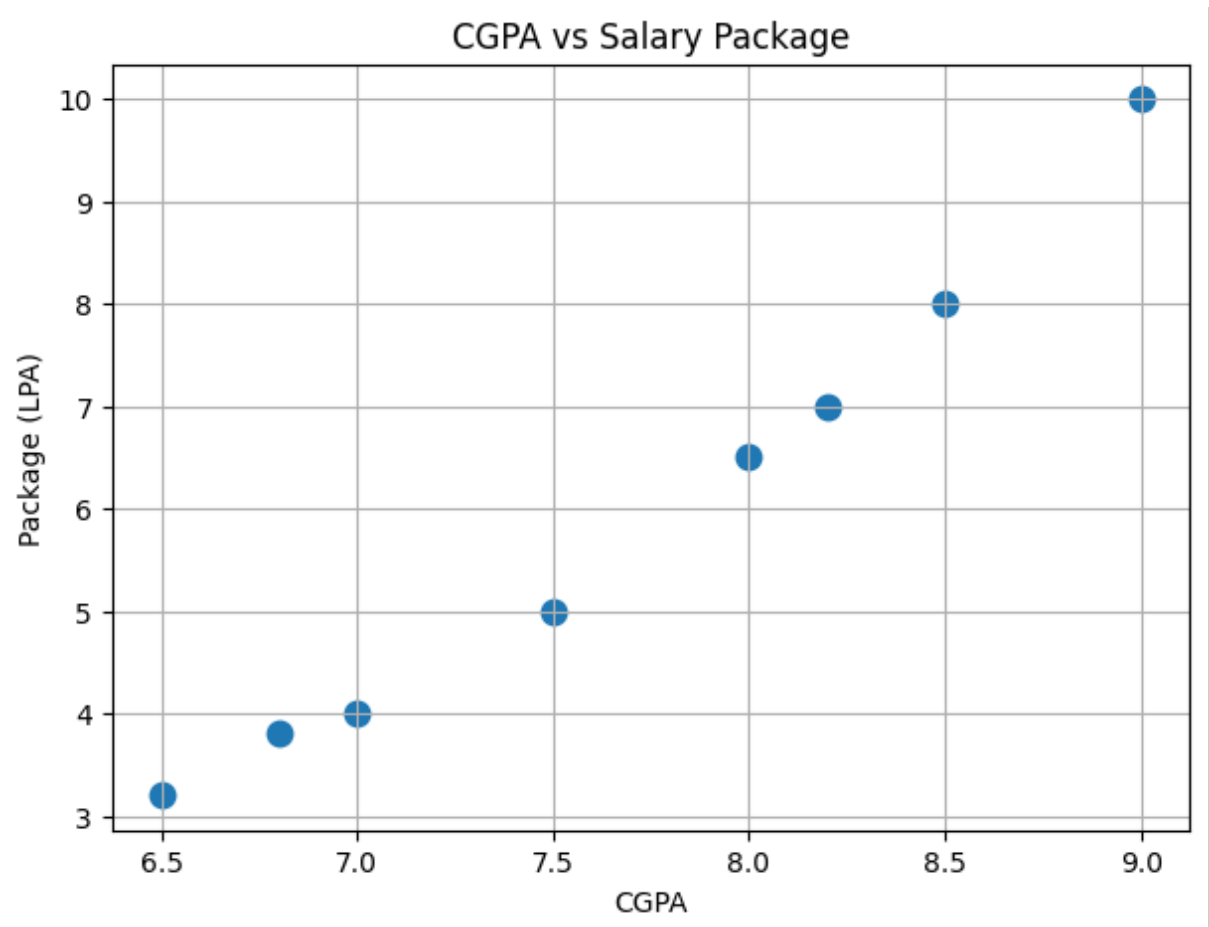
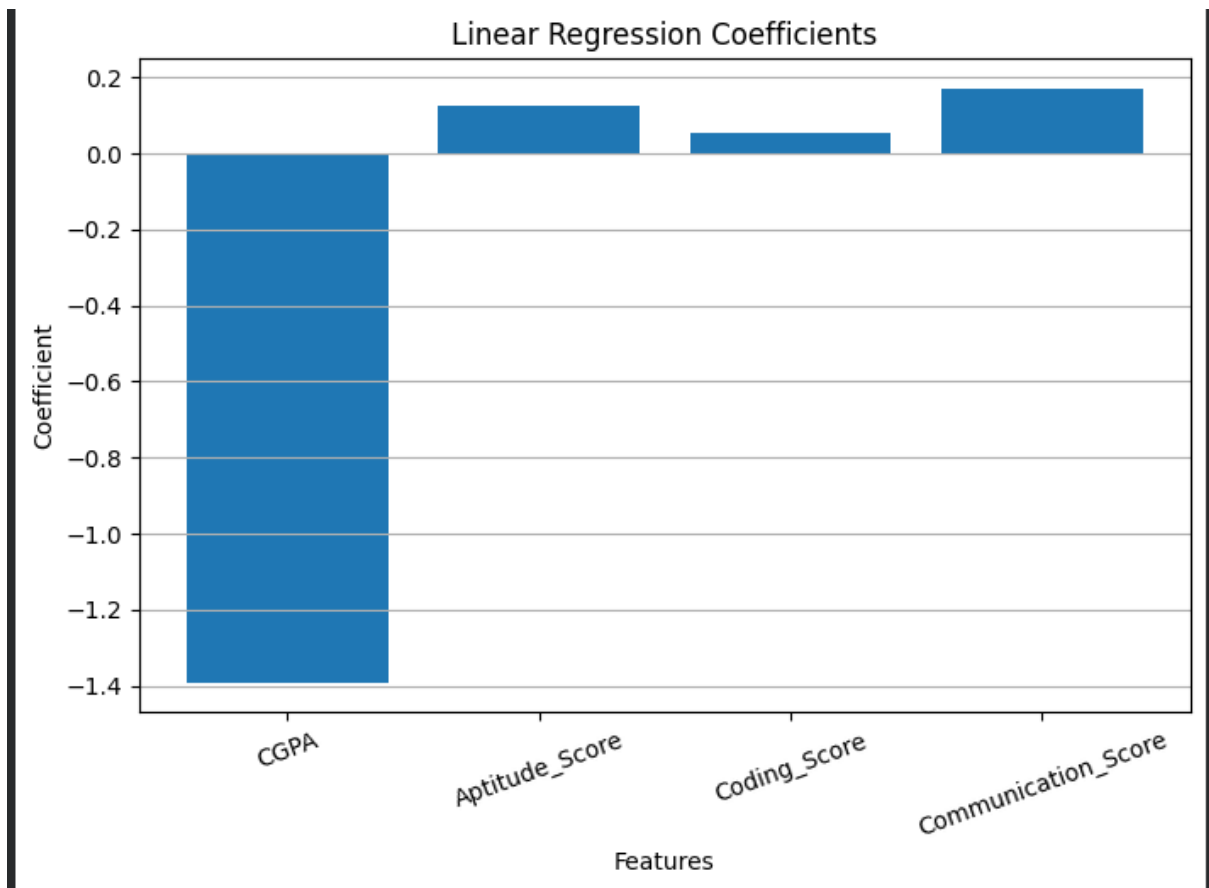
Intercept: -9.3905

New Student:

	CGPA	Aptitude_Score	Coding_Score	Communication_Score
0	8.1	82	85	78

Predicted Salary Package: 7.63 LPA





Question 10: Employee Attrition Prediction Using Logistic Regression

An HR department wants to predict whether an employee will **leave** the company or **stay**.

Employee	Experience	Satisfaction Score	Overtime Hours	Salary Increment %	Leave
E1	2	8	2	12	0
E2	3	7	4	10	0
E3	1	4	10	5	1
E4	5	6	6	8	0
E5	6	3	12	4	1
E6	8	2	14	3	1
E7	4	7	3	11	0
E8	7	4	11	5	1

Here,

Leave = 1 means employee may leave.

Leave = 0 means employee may stay.

Tasks

- Create the dataset using Python.
- Identify features and target variable.
- Split into training and testing data.
- Implement **Logistic Regression**.
- Predict attrition for test employees.
- Predict attrition for a new employee:

Experience	Satisfaction Score	Overtime Hours	Salary Increment %
5	4	9	6

- Evaluate using **accuracy, confusion matrix, precision, recall, and F1-score**.
- Explain how the HR team can use this prediction to reduce employee attrition.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score,
f1_score, ConfusionMatrixDisplay
```

```
data = {
    "Employee": ["E1", "E2", "E3", "E4", "E5", "E6", "E7", "E8"],
    "Experience": [2, 3, 1, 5, 6, 8, 4, 7],
```

```

    "Satisfaction_Score": [8, 7, 4, 6, 3, 2, 7, 4],
    "Overtime_Hours": [2, 4, 10, 6, 12, 14, 3, 11],
    "Salary_Increment": [12, 10, 5, 8, 4, 3, 11, 5],
    "Leave": [0, 0, 1, 0, 1, 1, 0, 1]
}

df = pd.DataFrame(data)

print(df)

X = df[["Experience", "Satisfaction_Score", "Overtime_Hours", "Salary_Increment"]]
y = df["Leave"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("\nActual Leave:")
print(y_test.values)

print("\nPredicted Leave:")
print(y_pred)

print("\nLeave Probability:")
print(np.round(y_prob, 2))

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
cm = confusion_matrix(y_test, y_pred)

print("\nModel Evaluation:")
print("Accuracy:", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1 Score:", round(f1, 4))

print("\nConfusion Matrix:")
print(cm)

print("\nLogistic Regression Coefficients:")
for feature, coefficient in zip(X.columns, model.coef_[0]):
    print(feature, ":", round(coefficient, 4))

```

```

print("\nIntercept:", round(model.intercept_[0], 4))

new_employee = pd.DataFrame({
    "Experience": [5],
    "Satisfaction_Score": [4],
    "Overtime_Hours": [9],
    "Salary_Increment": [6]
})

new_prediction = model.predict(new_employee)
new_probability = model.predict_proba(new_employee)[0][1]

print("\nNew Employee:")
print(new_employee)

print("\nLeave Probability:", round(new_probability * 100, 2), "%")

if new_prediction[0] == 1:
    print("Predicted Status: Employee May Leave")
else:
    print("Predicted Status: Employee May Stay")

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Stay", "Leave"]
)

disp.plot(values_format="d")
plt.title("Confusion Matrix - Employee Attrition")
plt.show()

plt.figure(figsize=(8, 5))
plt.bar(X.columns, model.coef_[0])
plt.xlabel("Features")
plt.ylabel("Coefficient")
plt.title("Logistic Regression Coefficients")
plt.xticks(rotation=20)
plt.grid(axis="y")
plt.show()

plt.figure(figsize=(7, 5))
plt.scatter(df["Satisfaction_Score"], df["Leave"], s=80)
plt.xlabel("Satisfaction Score")
plt.ylabel("Leave")
plt.title("Satisfaction Score vs Employee Attrition")
plt.yticks([0, 1], ["Stay", "Leave"])
plt.grid(True)
plt.show()

```


Output:

Employee	Experience	Satisfaction_Score	Overtime_Hours
0	E1	2	8
12			2
1	E2	3	7
10			4
2	E3	1	4
5			10
3	E4	5	6
8			6
4	E5	6	3
4			12
5	E6	8	2
3			14
6	E7	4	7
11			3
7	E8	7	4
5			11

Leave
0
0
1
0
1
0
1
1
0
1

Actual Leave:
[0 1]

Predicted Leave:
[0 1]

Leave Probability:
[0.32 1.]

Model Evaluation:
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

Confusion Matrix:
[[1 0]
[0 1]]

Logistic Regression Coefficients:
Experience : 0.0164
Satisfaction_Score : -0.2434
Overtime_Hours : 0.5315
Salary_Increment : -0.4229

Intercept: 0.8191

New Employee:

	Experience	Satisfaction_Score	Overtime_Hours	Salary_Increment
0	5	4	9	6

Leave Probability: 89.79 %

Predicted Status: Employee May Leave

